Vula OS Group
Soko
Internet-Draft
July 2026
Intended status: Standards Track (proposal)

Trade, Routing, Attestation, Custody & Trust (TRACT)

draft-tract-00

Abstract

TRACT is an open protocol for decentralized commerce: goods, services, rentals, subscriptions, and the delivery and settlement around them, between self-sovereign identities with no marketplace operator. A seller's catalogue is a signed, content-addressed feed the seller alone controls; an offer is a claim by one seller over a product record any number of sellers may share, so a single global product view emerges from content addressing rather than from a registrar. Orders are sealed messages between the parties, never published. TRACT adopts the DMTAP substrate (identity, feeds and blobs, sync, infrastructure roles, wake) unchanged and adds the commerce spine: a four-axis offer model covering goods and services alike, buyer-side carts and multi-seller checkout, published carrier and distributor rate cards from which routing is computed locally, purchase-attested reviews with no global score, jurisdiction as a first-class field, and a payment seam that names no provider. It composes existing standards (RFC 8949 CBOR, RFC 9052 COSE, RFC 8032 Ed25519, RFC 5545 iCalendar availability, ISO 3166/4217, schema.org product vocabulary, Incoterms, GS1 identifiers where present); the novelty is the composition, not new cryptography.

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of an open, royalty-free specification. Internet-Drafts are working documents; this document is a proposal and is *not* an RFC — an RFC number is assigned only upon publication by the relevant standards body. It may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use this document as reference material or to cite it other than as "work in progress."

Requirements Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, they appear in all capitals, as shown here.

Table of Contents

0. Overview & Architecture

0.1 Goals

TRACT is a protocol for **commerce between self-sovereign identities** — goods, services, rentals, subscriptions, and the delivery and settlement around them — with **no marketplace operator**, no registrar, and no token. A seller's catalogue is a signed feed the seller alone controls; a buyer's cart is state the buyer alone holds; the network in between stores nothing durable and adjudicates nothing.

Concretely, TRACT MUST provide:

1. **Sovereign selling** — a keypair is a store. No account with any platform is required to *be* a seller, and no party can delist, suspend, or edit a seller's catalogue (§1, §2).
2. **One product, many sellers** — a product record is content-addressed, so two sellers publishing the same record converge on the same address *by construction*. "Who sells this?" is a derived index anyone can rebuild, never a registry anyone runs (§2). **This is the mechanism, not yet a demonstrated solution** — independent publishers describing the same physical product do not produce identical bytes, and no deployed system achieves cross-publisher product identity without a licensed registry (§21.2).
3. **One shape for every trade** — goods, services, rentals, bookings, subscriptions, licences and made-to-order work all express as one four-axis offer, without a plugin per category (§3–§5).
4. **One cart across independent sellers** — a buyer assembles a cart spanning sellers who have never heard of each other, and checks out once (§6, §7).
5. **Delivery computed, not brokered** — carriers, distributors and peer couriers publish rate cards as public objects; routing and consolidation are computed **on the buyer's own node** from those objects, so no party sees the whole cart (§8).
6. **Settlement without a payment monopoly** — a provider-agnostic seam carries signed payment attestations; the protocol never holds funds and names no provider (§9).
7. **Trust without an authority** — reviews are signed objects proven against real purchases, and ranking is derived data any node computes. There is no global score, because computing one requires a party that aggregates and ranks (§10).
8. **Lawful by construction, worldwide** — jurisdiction, tax anchors and the responsible party are machine-readable fields on every offer and order, not an afterthought bolted onto a platform's terms of service (§11).
9. **Future-proofing** — crypto-agility and transport independence inherited from the substrate; standards reuse everywhere a standard exists (§20).

0.2 What TRACT is not

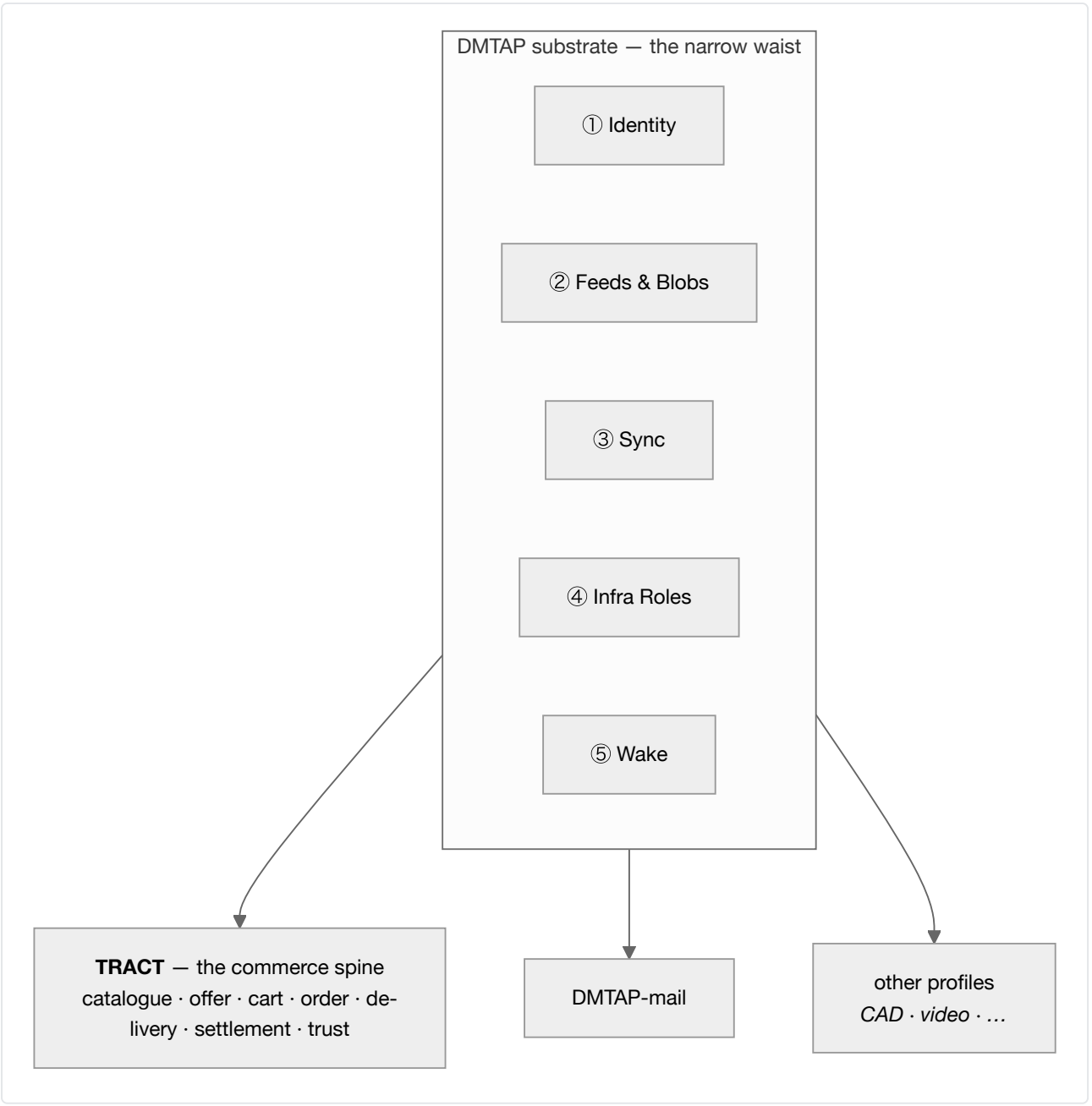
- **Not a marketplace.** There is no operator that lists sellers, ranks them, takes a cut, or can remove them. Where this document says "index" it always means derived, rebuildable data (§2.6).
- **Not a payment network.** TRACT specifies no rail, no currency, no token, and no ledger. It specifies a *seam* (§9) and the attestations that cross it.
- **Not a courier.** TRACT computes routes over published rate cards; it moves nothing.
- **Not an arbiter.** Where a dispute needs an adjudicator with power to seize funds, TRACT names the one role that may hold that power and confines it (§0.4, §9.6) rather than pretending the protocol can settle it.
- **Not a new cryptosystem.** Every primitive is inherited from the DMTAP substrate (§0.3), which in turn profiles existing RFCs.

0.3 TRACT stands on the DMTAP substrate

TRACT is not a new stack. It **adopts** the five substrate capabilities defined in the DMTAP substrate directory, under that directory's à-la-carte adoption rule — *if a product implements a capability's function, it MUST speak that capability's spec* — and adds only the commerce spine on top.

#	Substrate capability	What TRACT uses it for
①	**Identity** — keypair `IK`, `DeviceCert`, DNS `name→key`, key transparency, 8-word key-name	seller, buyer, courier, distributor and gateway identity (§1)
②	**Feeds & Blobs** — signed append-only per-author feeds, plaintext content-addressed blobs, servable over plain HTTPS	catalogues, offers, rate cards, capacity, reviews (§2, §8, §10)
③	**Sync** — signed CRDT op algebra, range-Merkle reconciliation	carts across a buyer's devices, inventory across a seller's replicas (§6)
④	**Infrastructure Roles** — announce/resolve, signaling, circuit relay, short-TTL content-blind mailbox, cache/pin	reachability for stores and buyers behind NAT (§1.5)
⑤	**Wake** — content-free, sender-blind push	waking a sleeping seller node when an order arrives (§1.5)

Sealed order messages (§7) ride the DMTAP **MOTE** (its §2) and its message kinds. TRACT allocates no new cryptography, no new hash construction, and no new signature framing.



The consequence worth stating up front: a seller's identity, a buyer's identity, and a mail identity are the *same key*. A shopper does not create an account to buy; they already have one, and it is theirs.

0.4 Roles, and the one operator class

Like the substrate, TRACT is **roles, not node types**. Every function below is a role of the same software, taken by whoever wants it, requiring nothing rarer than a machine that is up — with exactly one exception, and confining that exception is the structural claim of this document.

0.4.1 The roles anyone may take

Role	What it does	Why anyone can run it
Seller	publishes a catalogue feed; receives sealed orders	a keypair and a box that is up
Buyer	holds a cart; sends sealed orders; computes routing	a keypair
Courier	publishes a rate card; accepts consignments	a keypair; a bicycle is enough

Role	What it does	Why anyone can run it
Distributor	publishes storage/consolidation capacity; holds goods in transit	a keypair and space
Index	builds search/aggregation over public feeds	derived data; anyone may build one, none is authoritative (§2.6)
Cache / pin	serves content-addressed objects	inherited substrate role

None of these is sold, gatekept, or registered. An index in particular is **not** a marketplace: it holds no authority, and a disagreement between an index and a feed is always resolved in favour of the feed.

> **The gap between permission and practice (§21.3).** "Any node MAY build an index" does not mean many will. A content-addressed substrate offers no global index, so discovery is the **first** function to re-centralize: whichever index becomes economically dominant becomes a de facto content-policy gatekeeper, regardless of what this document permits. That happened to the closest deployed relative of this design, and the largest live decentralized-commerce network avoided it only by adopting a central approval-gating registry (§21.4). No protocol rule here prevents it. Multiple competing indexers with verifiable completeness or censorship proofs are a candidate answer with **no deployed precedent** (§21.8). This is the weakest load-bearing claim in the document and is marked as such rather than defended.

0.4.2 The gateway — the only operator class

A **gateway** is a role like the others, with one difference that defines it: it is the **only** function in TRACT that requires **scarce resources** — a domain with TLS and uptime, and, where it settles payments, a payment-provider relationship, a money float, and jurisdiction-specific licensing. None of that can be derived from a keypair or provisioned reciprocally.

A gateway does two things, and they bundle because the same commercial and legal standing underwrites both:

- **Storefront** (§12) — renders signed catalogue objects into ordinary HTML over ordinary HTTPS, so a shopper with no TRACT client and no keypair can browse and buy. It offers stores a subdomain and accepts custom domains.
- **Settlement and escrow** (§9.6) — holds funds between order and delivery under its own payment-provider relationship, and rules on release or refund.

It is **not** the catalogue (that is the seller's feed), **not** the index (derived, and anyone may build one), **not** the courier, **not** the reputation authority (§10.4), and **not** a holder of anyone's identity keys — ever. Two TRACT-native parties never need a gateway to transact; it exists for the legacy web and for buyers who want recourse.

Gateways are **permissionless to enter and competing**: a seller may list through several at once, and moving between them costs a DNS change, because the store *is* the feed.

0.4.3 Honest departure from DMTAP's structural claim

DMTAP confines its one operator class to legacy SMTP egress, and that class **self-extinguishes** as adoption grows. TRACT's does not:

- The **storefront** function is permanent, because browsers are permanent. A shopper without a keypair cannot verify a signature, so they trust the gateway rendered honestly. That is a real trust downgrade and it is disclosed as one (§12.6), mitigated but not removed by the fact that any node can re-render the same store and be compared byte-for-byte.

- The **escrow** function is permanent, because holding money for strangers is a licensed activity and physical custody cannot be made trustless (§9.6.4).

TRACT is therefore **structurally less pure than DMTAP**, and this document says so rather than discovering it later. What is preserved is that the class is *one*, entered permissionlessly, competed for, chosen per-order by both parties, replaceable without loss, and never in possession of identity keys.

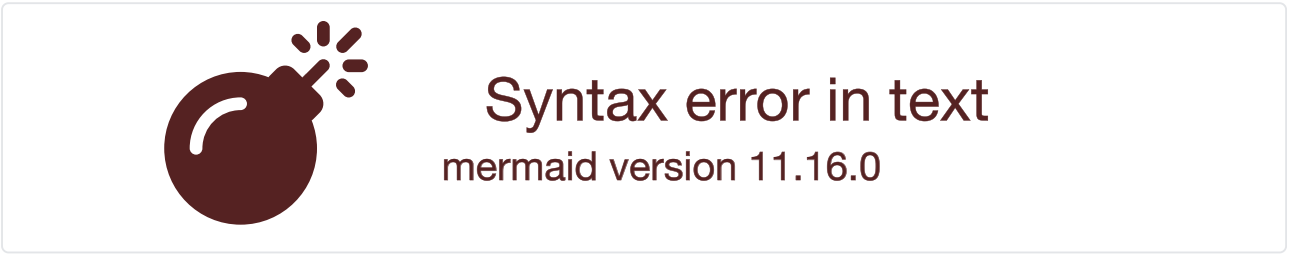
0.5 The two quadrants — what is public, what is sealed

The substrate's Feeds capability provides authenticity **without** confidentiality; its MOTE provides confidentiality **and** authenticity. TRACT splits commerce along exactly that seam, and the split is a hard rule, not a convention:

Public — signed, content-addressed, irrevocable	Sealed — encrypted, per-party, deletable
product records, offers, prices	orders, order lines
availability and stock signals	buyer name, address, contact
carrier and distributor rate cards	payment references and receipts
storefront render bundles	consignment routing detail
reviews (pseudonymous, §10.3)	dispute correspondence

> **Normative (§0.5.1). No personal data may enter the public quadrant.** Published objects are irrevocable and content-addressed; a right to erasure cannot be satisfied against them. An implementation MUST NOT publish, plaintext-address, or serve any object containing personal data. Orders are sealed, always. The residual case — reviews, which are public by nature and signed by a person — is bounded in §10.3 and disclosed in §14 as an honest limit.

0.6 The shape of a trade



The buyer's node is the orchestrator. There is no party that sees the whole cart, and no party whose absence prevents the trade — except a gateway, when the parties chose one.

0.7 Where state lives

State	Location	Notes
Catalogue, offers, rate cards, capacity	**Seller / courier / distributor node** (the edge)	published as signed feeds
Cart, wishlist, purchase history	**Buyer node** (the edge)	CRDT across the buyer's own devices; survives any store closing
Orders	**Both endpoints**, sealed	never in the middle, never public
Product records	**Content-addressed swarm**	globally deduplicated; any holder may serve
Indexes, search, rankings	**Anywhere**	derived, rebuildable, never authoritative

State	Location	Notes
Funds in escrow	**A node in gateway mode**	the one irreducible operator function
Reachability (key → location)	**Substrate mesh / HTTPS**	signed, TTL'd, self-republished

Every row but the escrow row is either edge state or derived data. That is the invariant the rest of this document protects.

The liveness caveat, stated where the table is rather than late (§21.5). Durability at the edges covers *orders* — a sender's node retries, and an offline seller's orders wait. It does not cover *catalogues*. A seller whose node is offline is not slow; they are **invisible**. The one deployed system closest to this design measured a ~22-day median listing lifetime and whole catalogues disappearing when a merchant node departed. Any deployment therefore depends on third-party caching or pinning of public objects for sellers who are not always on — and unpaid replication is precisely what that system did not get. Whether pinning needs an incentive, and whether that incentive creates another operator, is open (§21.8).

0.8 Document map

§	Title	Covers
§1	Actors & identity	seller/buyer/courier/distributor/gateway identity, reachability
§2	Catalogue	product records, the product-identity ladder, offers, variants, indexes
§3	Availability	stock, time slots, capacity, made-to-order
§4	Fulfilment	ship, collect, digital grant, perform-at, perform-remote, access, return
§5	Consideration	fixed, tiered, recurring, metered, deposit, quote/RFQ; tax
§6	Cart	buyer-side CRDT cart, live availability, holds and reservations
§7	Order	the sealed order object and its state machine
§8	Delivery	rate cards, legs, consolidation, distributors, local route computation
§9	Settlement	the payment seam, rail classes, escrow, the gateway's money role
§10	Trust	purchase-attested reviews, local ranking, why no global score
§11	Jurisdiction	the four anchors, scope declarations, tax and consumer-law fields
§12	Gateway	storefront rendering, domains, origin isolation, honest trust limits
§13	Analytics	privacy-preserving measurement; what a merchant gains and loses
§14	Anti-abuse	listing spam, review manipulation, Sybil, and the honest limits
§15	Conformance	profiles and the auditable fail-closed set
§16	Wire format	deterministic CBOR object definitions
§17	Errors & registries	the `ERR_TRACT_*` block and IANA-style registries
§18	State machines	offer, order, consignment, escrow
§19	Parameters	timeouts, limits, defaults
§20	References	normative and informative standards
§21	Grounding	what the evidence supports, and what it contradicts
§22	Erasure	the erasure-rights conflict, candidate mechanisms, and the residual left unresolved

0.9 Conventions & normative glossary

Requirement language. The key words MUST, MUST NOT, SHOULD, SHOULD NOT, MAY are to be interpreted as described in BCP 14 (RFC 2119, RFC 8174) when, and only when, in all capitals.

Glossary (normative). Defined once here, used with these meanings throughout.

- **product record** — a content-addressed public object describing *what a thing is*, not who sells it or for how much. Shared across sellers by construction (§2.2).
- **offer** — a signed announcement by **one** seller that it will supply a product record's subject on stated terms. An offer is always one seller's claim; a product record is nobody's (§2.4).
- **the four axes** — every offer declares **Item**, **Availability**, **Fulfilment**, and **Consideration** (§2.4.3). Together they express goods, services, rentals, subscriptions and bookings without category-specific machinery.
- **index** — derived, rebuildable aggregation over public feeds. **Never authoritative**; a disagreement with a feed resolves in favour of the feed (§2.6). The term *marketplace* is **not used** in this document for anything TRACT defines, precisely because an index is not one.
- **gateway** — **one sense only**: the operator-class role of §0.4.2 — storefront rendering and/or settlement and escrow. It never means an index, a courier, or a relay.
- **consignment** — physical goods in transit under someone else's custody: a courier leg or a distributor hold (§8.4).
- **leg** — one movement of goods between two places, priced by one rate card (§8.2).
- **rate card** — a signed public object from which a leg's price and transit estimate are **computed locally**, as opposed to quoted by a call to the carrier (§8.2).
- **place of supply** — the jurisdictional anchor derived from the **Fulfilment** axis, distinct from seller establishment, buyer residence, and delivery destination (§11.2). Getting these four confused is the most common commerce-tax error and this document keeps them separate by construction.
- **rail** — a settlement mechanism behind the payment seam, classified `CustodialReversible` or `NonCustodialFinal`. The class is part of the type because it changes the buyer's recourse (§9.2).
- **purchase attestation** — a signed proof that a review's author actually transacted, issued by the seller or by an escrow gateway (§10.2).
- **personal data** — as defined by the applicable regime (GDPR Art 4(1), POPIA §1, LGPD Art 5); for this document's purposes, any datum relating to an identified or identifiable natural person, **including a pseudonymous key that is linkable to one**.

1. Actors & identity

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

1.1 Scope

Who the parties are, how they are identified, and how they are reached. TRACT adopts substrate capability ① (Identity) unchanged and adds only the commerce-specific roles and disclosures.

1.2 One identity type, five roles

Seller, buyer, courier, distributor and gateway are roles a key takes by what it does, never separate account kinds a key applies for. The identity underneath every one of them is the substrate's `IK` plus an optional `DeviceCert` (§1.8) — the same construction a mail identity uses (§0.3).

Role	Becomes that role by
Seller	publishing a signed catalogue feed (§2)
Buyer	holding a cart and sending a sealed order (§6, §7)
Courier	publishing a rate card (§8)
Distributor	publishing a capacity record (§8)
Gateway	the one role needing more than a key — a domain, uptime, and, where it settles, a payment-provider relationship (§0.4.2)

Because the role is what a message does, and not a field anywhere records, there is no registration step — there is nothing to register with. No table anywhere maps a key to "seller" or "buyer"; a key is whichever of these it is currently acting as, and a verifier checking a signed object never has to ask which. A single key can be all five at once: a seller restocking from a wholesaler signs the purchase order with the identical key it publishes its own catalogue under, and nothing in the wire format (§16) distinguishes a "seller key" from a "buyer key" — there is one `identity-key` type (§16.3). Gateway is the exception only in what it costs to enter, never in what it is: the same key that runs a storefront can also be a buyer on somebody else's.

1.3 Seller disclosure

Several trader-traceability duties assume a seller's identity comes with a minimum disclosure attached to it, not just a key. TRACT gives an offer the fields those duties ask for; it does not decide which duties apply to a given trade — that is §11's job, and §11.4 lists the regimes it has to accommodate.

Field	What it answers
legal or trading name	who the seller claims to be
contact route	how a buyer or a regulator reaches them — not necessarily a physical address
establishment country	the seller-establishment anchor (§11.2)
registration reference, where the seller holds one	a company number, VAT number, or equivalent

A self-published claim of a trading name carries no more authority than any other field in an offer the seller alone signs — it is evidence of what the seller asserted, not proof of who they are. Whether that is sufficient to discharge a legal disclosure duty is exactly the kind of question §11 exists to answer and largely cannot yet: trader-traceability regimes specifically are one of the four legal questions a dedicated pass returned nothing verified on, across three attempts (§21.11). The field's presence in this section is not a compliance claim.

1.4 Buyer disclosure tiers

A buyer discloses more of themselves the closer they get to a transaction, and the tiers stay distinct rather than collapsing into one identity handed over on first contact.

Tier	What is attached	Seen by
Browse	nothing — fetching a product record, offer or rate card is an anonymous pull (§0.6)	nobody
Grant	the buyer's per-seller pseudonymous subkey (§1.5), enough to be recognised as a repeat visitor without a name	that one seller
Order	full detail — name, delivery address, contact — carried inside the sealed `Order` (§16.6)	that one seller, and only for that one order's lines

Nothing between browse and order is public: the grant tier's subkey is not a new object type, it is the same key a buyer already uses when returning to that seller's cart or leaving a review (§1.5), and the order tier's detail never leaves the sealed channel it arrives in (§0.5.1).

1.5 Per-store pseudonymous subkeys

A buyer who wants to be recognised by a seller they return to — a repeat-customer discount, a review with continuity, a support thread that doesn't start from zero — needs *some* linkability. Full identity would leak it to every seller at once; a fresh key per interaction would leak none, including to the seller who should have it. TRACT's answer is a subkey scoped to one seller: linked within that relationship, not across it. It is the same construction `Review.author` already relies on (§16.5.5) — a per-subject subkey, never the root `IK`.

Two ways to derive it, and the choice is a real trade-off rather than an implementation detail:

- **Deterministic** — the subkey is a function of the buyer's root key material and the seller's identity key. Nothing needs storing or backing up; the same subkey falls out again on a new device with the same root key. The honest cost: anyone who later learns the buyer's root key — a compelled disclosure, a seized device, a leaked backup — can recompute every per-seller subkey the buyer ever used and join their purchase history across every seller at once, retroactively. The unlinkability was never structural; it was the difficulty of the computation, and that difficulty is zero for the one party who holds the root key.
- **Stored random mapping** — a subkey generated independently per seller and kept in the buyer's own synced cart state (§6.2, substrate capability ③). Learning the root key reveals nothing extra about it. The cost lands on recovery: losing the mapping — not the root key, the mapping itself — severs the link to that seller's history, and a buyer restoring from a bare seed phrase gets a working identity but a stranger's standing with every seller they had a relationship with.

Either way, two sellers comparing customer lists cannot join them by subkey; only the buyer, holding the root key, can ever compute or has ever stored the correspondence.

1.6 Reachability

A seller, courier or distributor is reached by identity key, not by network address, which is what makes CGNAT irrelevant rather than a deployment obstacle to route around. The path is the substrate's own ladder (§0.3 ④): announce/resolve locates a current address for the key; a direct connection is tried first; where NAT or firewall traversal fails, a circuit relay carries the session without seeing its content; where the recipient is offline entirely, a short-TTL content-blind mailbox holds what was sent until it is collected.

That last step matters specifically for orders. A sealed order addressed to a sleeping seller's key does not fail and does not wait on the sender — it lands in the mailbox, and wake (§0.3 ⑤), a content-free, sender-blind push, is what can bring the seller's node back online to collect it. The same primitive substrate mail uses to wake a sleeping inbox wakes a sleeping storefront for exactly the same reason: neither the wake signal nor the mailbox operator learns anything about what is waiting.

1.7 The liveness asymmetry (§21.5, §21.9)

Reachability by key (§1.6) covers *orders*, not *catalogues*, and this section has to keep that distinction visible rather than let "reachable by key" imply both. A sleeping node still receives orders: the sender's node retains the retry queue and the mailbox-plus-wake path of §1.6 brings the recipient back. A sleeping node does **not** serve its catalogue while asleep — so an offline seller is not slow, they are invisible, and nobody else is obliged to serve their listings in their absence. The closest deployed relative of this design measured a ~22-day median listing lifetime and whole catalogues disappearing when a merchant node departed (§21.3). Third-party caching or pinning of public objects therefore moves from a convenience to a practical requirement for any seller who is not always on, and unpaid replication is exactly what that system failed to attract. Whether pinning needs an incentive, and whether that incentive creates another operator, stays open (§21.8).

1.8 Standards profiled

Ed25519 (RFC 8032), deterministic CBOR (RFC 8949 §4.2), COSE (RFC 9052), and the substrate's `DeviceCert`, key-transparency and key-name constructions. No new key type is introduced.

1.9 Open

- Whether a seller's legal-disclosure block (§1.3) belongs in the offer, the feed head, or both.
- Whether subkey derivation (§1.5) should be deterministic or a stored random mapping — the trade-off is stated above; which one the spec should mandate, or whether it leaves the choice to implementations, is not decided.
- Whether the grant tier (§1.4) needs its own signed disclosure object, or is fully satisfied by the per-seller subkey with no separate protocol object at all.

2. Catalogue

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

2.1 Scope

Product records, offers, the product-identity ladder, variants, and the rules that keep indexes from becoming authorities.

2.2 The split, and why it is mechanical rather than conventional

A **product record** describes what a thing is; an **offer** is one seller's claim to supply it. Because the substrate content-addresses public blobs over plaintext, two sellers publishing the same record converge on the same address by construction, and the swarm stores it once. The global product view is therefore an emergent consequence of hashing, not a registry.

2.2a How strong the convergence claim actually is (§21.2)

Weaker than §2.2 sounds, and §21.9 obliges this section to say so rather than let a reader infer otherwise.

Convergence is trivially true for identical bytes and says nothing about the real case: two shops describing the same shoe. A July 2026 literature pass found **no deployed system achieving cross-publisher product identity without a licensed registry**, and the one candidate for permissionless crawl-derived resolution was

refuted 0-3 under adversarial verification. The two models that exist in the field are the only two: a permissioned monopoly namespace (GS1 GTIN — licensed rather than sold, fee-bearing, issued through national member organisations) and a purely nominal string with no issuer or uniqueness guarantee (schema.org `productGroupID`). Nothing in between is deployed.

So the content address is a sound **mechanism** carrying an **unproven** claim. The canonicalisation rules of §2.3 — not the hashing — are the load-bearing part of this section, and §2.5 keeps near-duplicate resolution listed as open rather than implied-solved.

2.3 What this section will specify

- `ProductRecord` and `Offer` object shapes.
- **Canonicalisation**: the normalisation applied before addressing, since convergence is only useful to the extent independent publishers can actually produce identical bytes. This is the hard part of the section.
- The **identity ladder** — content address (floor, zero authority) → claimed external identifiers (advisory, unverified, squattable) → manufacturer-signed record (authority = the brand).
- **Variants**: product groups, varies-by axes, and per-variant records.
- **Bundles and kits**, including components published by other sellers.
- The **index rule**: derived, rebuildable, never authoritative; disagreement resolves toward the feed; no protocol mechanism exists by which an index can delist a seller from the network.
- **The gap between permission and practice** (§21.3, §21.9). "Any node may build an index" does not mean many will. A content-addressed substrate offers no global index, so discovery is the *first* function to re-centralize: whichever index becomes economically dominant becomes a de facto content-policy gatekeeper regardless of what this document permits. That is what happened to the closest deployed relative of this design, and the largest live decentralized-commerce network avoided it only by adopting a central approval-gating registry (§21.4). **No rule in this document prevents it.** Multiple competing indexers with verifiable completeness or censorship proofs is the candidate answer, and it has no deployed precedent (§21.8). This is the weakest load-bearing claim in the specification and is marked as such rather than defended.

2.4 Standards profiled

schema.org product vocabulary (`Product`, `Offer`, `ProductGroup`, `hasVariant`, `variesBy`) for the data model, so existing merchant feeds map in by translation. GS1 identifiers (GTIN, MPN) are supported as **claims only** — issuance is gated and fee-bearing, so a spec that depended on them would import a centralization point and a cost barrier.

2.5 Open

- Near-duplicate resolution. The address floor is exact-match; merging almost-identical records is an index-side heuristic, and heuristics differ between indexes. Whether the spec should recommend one, or deliberately leave it to differ, is undecided.
- Bootstrapping manufacturer signatures when most brands will not participate early.

3. Availability

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

3.1 Scope

The first of the four offer axes: when a thing is available, and how much of it there is. §16.5.2 already defines the grammar; this section is the reasoning behind each variant and the rule for evaluating one.

3.2 Variants

Variant	Carries (§16.5.2)	Covers
`count`	a `StockSignal` band	physical or digital stock, exact or banded
`time-slots`	an RFC 5545 payload, slot length in minutes	bookable appointments
`capacity-per-interval`	a capacity number, an RFC 5545 recurrence	seats, tables, room-nights
`unlimited`	nothing	digital goods with no bound
`made-to-order`	lead days	built or provisioned after the order, not held in stock

Five variants, one grammar. A haircut and a restaurant table are both bookable, but a haircut consumes an appointment slot and a table seats a party against a nightly cap — `time-slots` and `capacity-per-interval` are distinct variants because those are different questions, not the same question asked twice.

3.3 The stock signal is a band, not a number

```
StockSignal = exact(n) / in-stock / low / out-of-stock
```

A seller may publish `exact(n)` when it chooses to, but the count variant does not require it, for two reasons that both matter:

- **Exact stock is commercially sensitive.** A competitor watching a public feed can infer sell-through rate, restock cadence, and even unit economics from a bare number changing over time. Browsing a catalogue does not need that number; it needs to know whether the thing can be bought.
- **A band degrades honestly; an exact number degrades into a lie.** Stock moves between publishes. `in-stock` stays true across a wide range of underlying counts, so it survives the gap between one publish and the next. `exact(4)` is precisely correct at the moment it is signed and increasingly wrong afterwards — and it is wrong with the same apparent confidence throughout. A band's imprecision is disclosed by its shape; a stale exact count's imprecision is not disclosed at all.

`out-of-stock` and `low` are distinct from each other for the same reason `made-to-order` is distinct from `out-of-stock` (§3.6): a buyer deciding whether to add something to a cart needs to know whether waiting a moment might work, not just whether it works right now.

3.4 Time slots — profiling RFC 5545

`time-slots` carries an opaque RFC 5545 payload (`VAVAILABILITY` / `VFREEBUSY`) plus a slot length in minutes, rather than a bespoke schedule grammar. The reasoning is the same one §5.6 makes for recurring consideration: a seller who already runs calendar software should publish from it directly, not maintain a second, TRACT-specific source of truth that drifts out of sync with the first.

The buyer's node parses the payload locally against the slot length to enumerate candidate slots — there is no slot-listing endpoint to call. What the payload cannot express is which slots other buyers have since taken; §3.9 covers why that is a staleness problem, not a grammar problem.

3.5 Capacity per interval

`capacity-per-interval` carries a unit count and an RFC 5545 recurrence describing the intervals it applies to — 40 covers per sitting, three sittings a night. This is not `time-slots` with a number attached: a time slot is claimed whole by one booking, where a capacity interval is shared by many bookings up to its cap. Modelling a restaurant table as a time slot would force one party to occupy the whole sitting; modelling a haircut as capacity-per-interval would allow two people to book the same ten minutes.

3.6 Made-to-order is not out-of-stock

`made-to-order` carries a lead-time figure and nothing else — no stock number, no slot, no capacity, because none of those describe it. The thing is available. It simply is not sitting in a warehouse yet, and that is a different fact from having none.

Conflating the two is why lead-time products display badly on retail platforms built around a stock count: a made-to-measure suit or a build-to-order machine gets forced into `in stock` (wrong — it does not exist yet) or `out of stock` (wrong — it is buyable right now, just not instantly). Neither label is honest, and a buyer who wanted to know "how long" was never asked the right question. `made-to-order` exists so that question has a field.

§19.2's `FULFIL_TIMEOUT` reads this figure directly: it cannot be one protocol-wide number because a made-to-order lead time and an instant digital grant have nothing in common, so the availability axis supplies the floor a buyer may expect to wait before cancelling becomes reasonable, rather than the protocol asserting a duration it cannot know.

Whether the lead-time figure is counted in calendar days or working days is not settled by the grammar — see §3.12.

3.7 Evaluating availability locally

The buyer's node does this work; nothing is queried live.

Variant	Local evaluation
`count`	compare the requested quantity against the band. `exact(n)` supports an exact check; `in-stock` / `low` support only "probably yes, ask to be sure"; `out-of-stock` blocks the add.
`time-slots`	expand the RFC 5545 payload against the slot length into candidate slots and present them.
`capacity-per-interval`	expand the recurrence into intervals and show the published cap per interval — not how much of the cap is already taken by other buyers.
`unlimited`	nothing to evaluate.
`made-to-order`	add the lead days to the order date to show an expected dispatch date, feeding §19.2's `FULFIL_TIMEOUT` floor.

None of this evaluation contacts the seller. That is the point of publishing the object at all — but it also means every row above is read from whatever copy of the offer the buyer's node last fetched, which is the subject of §3.9.

3.8 What happens when published availability is stale

An `Offer` carries one freshness signal: its own `published` timestamp (§16.5.2, key 6). Availability has no timestamp of its own — a stale `Offer` and a stale `Availability` are the same event, because they are the same object. There is no push notification for an availability change specifically; substrate capability ⑤ wakes a seller's node for incoming orders, not a buyer's node for outgoing price or stock changes. Freshness

is bounded only by how often the buyer's client re-fetches the seller's feed, no more often than `FEED_POLL_MIN` (§19.6) but with no upper bound the protocol enforces.

The consequence: a buyer can add something to a cart that sold out between the last fetch and now. This is not treated as a protocol defect to be engineered away — §3.3's whole argument is that a band already discloses this uncertainty rather than hiding it. What resolves the gap is not a tighter poll interval; it is that the seller's `placed → accepted / declined` step (§18.3) is authoritative and the browsing-time signal is not. An offer that turned out to be wrong is declined there, explicitly, rather than silently honoured against stock that no longer exists.

3.9 A signal, not a reservation

Nothing in this section commits stock. `Availability` is what a seller publishes and what a buyer's node reads to decide whether adding something to a cart is worth attempting — it is input to a decision, not a hold on the outcome.

§6 is where a hold actually exists: a seller's bounded-counter inventory (§6.2, §6.2a) is what guarantees that concurrent sales never exceed real stock, and it operates beneath what gets published here. A buyer's node never sees counter state directly, only the band this section defines. Reading `Availability` as a reservation would be a category error in both directions — it would credit the band with a guarantee only the counter provides, and it would blame the band for a staleness gap that the counter, not the signal, is responsible for closing.

3.10 Standards profiled

RFC 5545 `VAVAILABILITY` / `VFREEBUSY` for `time-slots`; RFC 5545 recurrence rules for `capacity-per-interval`'s interval definition. Time zones per RFC 5545 / IANA tzdata.

3.11 Open

- Whether published availability bands need a normative vocabulary or stay seller-defined.
- Whether `Availability` needs its own freshness parameter, distinct from the offer's `published` timestamp and from `RateCard`'s `RATE_CARD_MAX_AGE` (§19.6) — stock and slots plausibly move much faster than price, and currently nothing distinguishes "this offer is a day old" from "this stock band is a day old" because they are the same field.
- Whether `made-to-order` lead days are calendar days or working days. The reference implementation documents working days; the grammar carries a bare integer and does not encode the distinction.
- Whether the capacity number in `capacity-per-interval` is meant to be read as remaining capacity or total capacity once bookings exist against an interval — the same signal-versus-reservation question as `count`, unresolved for this variant specifically.

4. Fulfilment

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

4.1 Scope

The second offer axis: how the thing reaches the buyer. This is not only a logistics question — it is the **only** object that knows where a supply happens, and §11.2 derives the tax anchor from it and from nothing else.

4.2 Variants

Variant	Carries (§16.5.2)	Covers
`ship`	destination countries	carried to an address
`collect`	a `PlaceRef`	buyer picks up
`digital-grant`	nothing	download, licence key
`perform-at-place`	a `PlaceRef`	the venue for a haircut, an event
`perform-remote`	nothing	consulting over video
`access-grant`	a `PlaceRef` or nothing	a gym membership, or a streaming subscription — same variant
`return-required`	a `PlaceRef`, a term in days	a rental or hire

Each `Offer` carries exactly one `Fulfilment` value (§16.5.2, key 3) — the union is a choice, not a set. §4.8 covers what that means for a seller who genuinely offers more than one fulfilment mode for the same item.

4.3 The place-of-supply derivation table

Fulfilment variant	Anchor	Why
`ship`	delivery destination	the goods physically arrive there; import VAT/GST and duty regimes key off where they land
`collect`	the stated place	the buyer takes possession there, regardless of where either party is established
`perform-at-place`	the stated place	admission and physically-performed services are generally taxed where the performance happens
`return-required`	the stated place	the same reasoning as `collect` — the item changes hands there
`perform-remote`	buyer residence	there is no physical venue to anchor to
`digital-grant`	buyer residence	nothing physical happens anywhere
`access-grant`	the stated place, if named; otherwise buyer residence	the field name is shared by two economically different cases, and the anchor has to follow whichever one this instance actually is

This is the forcing example §0.1 and §11.2 both cite: an event held in one country, sold by a seller established in a second, to a buyer resident in a third. Only the venue in the `Fulfilment` object answers the question; neither party's country does, and averaging or defaulting to either one is simply wrong.

4.4 Why the anchor is derived, never supplied

§11.2 states the rule this section has to honour: place of supply is **computed from** the `Fulfilment` object, and it is not a separate field an implementation fills in alongside it. This is stricter than it sounds, and it is stricter on purpose.

An earlier implementation took place of supply as its own parameter, populated independently of the fulfilment details rather than derived from them. Once the two could be set independently, nothing checked that they agreed — and in practice the anchor defaulted to the seller's own establishment, because that is the natural default in ordinary billing code, not because anyone decided a haircut performed abroad should be taxed at home. The system returned a country. It was wrong, and nothing about the output looked wrong: no error, no missing field, just a confidently incorrect jurisdiction.

Deriving the anchor as a pure function of the `Fulfilment` value — §16.5.2's grammar comment, and `Fulfilment::place_of_supply_kind` in the reference implementation — removes the second parameter entirely. There is nothing left for the anchor to disagree with, because there is nothing else it could be computed from. This is also why neither `Offer` nor `Order` carries a place-of-supply field of its own (§16.5.2, §16.6): adding one back would reopen exactly this failure mode.

4.5 Handover: what counts, and who signs it

§18.4 already states the general rule for custody: a handoff is signed by the party **taking** custody, not the party giving it up, because a chain attested only by the sender proves someone tried, and attested by the receiver proves something actually moved. Fulfilment variants differ in whether a custody chain exists at all.

Variant	What constitutes handover	Evidence
`ship`	the carrier takes custody at first-leg pickup, beginning the consignment chain (§18.4 `created → accepted`, signed by the receiving custodian); final handover is the last leg's `delivered` transition	carrier or recipient signature, per §18.4
`collect`	the buyer takes physical possession at the stated place	no consignment leg exists — see below
`digital-grant`	the grant (download, licence key, credential) is transmitted	signed by the seller at transmission
`perform-at-place`	the service is completed at the venue	—
`perform-remote`	the remote session or deliverable is completed	—
`access-grant`	access is issued, and for a term, remains valid until it lapses	signed by the seller at issuance
`return-required`	two handovers: outbound at the start of the term, inbound return at or before it	—

The gap this table exposes. §18.3's order state machine signs the generic `fulfilling → delivered` transition as "carrier or seller". That is a good fit for `ship`, where a carrier's proof-of-delivery is independent third-party evidence. It is a weaker fit for `collect`, `perform-at-place`, `perform-remote` and `access-grant`, where there is no carrier and the signature available is the seller's own claim that handover happened — exactly the kind of unilateral assertion the signed-transition model exists elsewhere to avoid. Whether those variants should instead require the buyer's counter-signature to reach `delivered` is not resolved here; see §4.10.

4.6 Incoterms 2020: risk and cost transfer for shipped goods

Incoterms 2020 govern a different question from §4.3's anchor, and the two are easy to conflate the same way seller/buyer country and place of supply are (§11.2): Incoterms fix **when risk and cost** pass from seller to buyer on a shipped good, not **where the supply is taxed**. A `DAP` shipment and an `EXW` shipment to the same destination have the same place of supply and different points at which the buyer bears loss in transit.

The `Fulfilment::Ship` variant carries only destination countries (§16.5.2) — it has no field for which Incoterm applies. That term, where it matters, currently sits at the leg or order level rather than the offer's fulfilment axis; whether it needs a wire field of its own is open (§4.10).

4.7 Return terms for rentals

`return-required` carries a place and a term in days and nothing else. What happens if the item comes back damaged, or late, is not a fulfilment-axis question — it is a dispute over an order already placed, resolved through the order and escrow machinery (§7, §18.5) rather than encoded as a condition on the offer. The natural pairing is with `DepositBalance` consideration (§5.8): a deposit taken at order time is the mechanism by which a late-return or damage claim is actually made whole, and the fulfilment axis's job stops at stating where and for how long, not at what enforces it.

4.8 One item, several ways to get it

A seller who genuinely offers both `collect` and `ship` for the same item — common, not an edge case — cannot express that as a single `Offer`, because `Fulfilment` is a one-of union and an `Offer` carries exactly one value of it (§16.5.2, key 3). The way to express it under the current grammar is two `Offer` objects: same `Item`, same `Availability`, same `Consideration`, differing only in `Fulfilment`. Each is independently signed and independently withdrawable (§18.2). A buyer's node presents both against the same underlying product and lets the buyer's fulfilment choice pick which offer — and therefore which anchor (§4.3) — the resulting order binds to.

This replaces the earlier framing of "multi-variant offers" with the grammar's actual shape: there is no offer-level alternative-fulfilment construct today, only multiple offers that happen to share everything but the fulfilment axis. Whether that repetition is worth a dedicated grouping construct is open (§4.10).

4.9 Standards profiled

Incoterms 2020 for the risk/cost transfer point on shipped goods. ISO 3166-1 for places (`PlaceRef`, §16.5.2).

4.10 Open

- Whether `digital-grant` needs a licence-terms sub-object, or defers entirely to §5's consideration axis.
- Whether an Incoterm needs its own wire field on `Ship`, or remains a leg/order-level convention outside the offer (§4.6).
- Whether fulfilment variants without a carrier (`collect`, `perform-at-place`, `perform-remote`, `access-grant`) should require the buyer's counter-signature to reach `delivered`, rather than accepting the seller's signature alone (§4.5).
- Whether repeated same-item, different-fulfilment offers (§4.8) warrant a first-class grouping construct, given the grammar currently expresses the case only as separate, independently signed objects.

5. Consideration

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

5.1 Scope

The third offer axis: what is paid, on what schedule, and how tax attaches to it.

5.2 Variants

Variant	Carries (§16.5.2)	Covers
`fixed`	one `money`	a single price
`tiered` / `volume`	`[+ PriceTier]` (min_qty, unit_price)	cheaper per unit above a threshold
`recurring`	`money`, an RFC 5545 `RRULE`	subscriptions
`metered`	a dimension string, a unit `money`	usage billed after the fact
`deposit + balance`	two `money` values	part now, remainder on delivery or completion
`quote-required`	nothing	RFQ, and B2B contract pricing

5.3 Money is minor units and a currency code, never a float

```
money = { 1 => int, 2 => currency }   ; minor_units, ISO 4217
```

This matters more inside `Consideration` than it would in an ordinary API, because a `Consideration` value is either signed directly (inside a published `Offer`) or carried into a sealed `Order` whose transitions are themselves signed (§18.3). A float invites a value like 19.999999999998 to enter through a UI slider or a percentage calculation, and once that value is signed, the signature covers the wrong number — there is no later step at which it can be corrected, only a new object that supersedes the old one and a gap in between where the wrong figure was the authoritative one. Integer minor units foreclose the error at the type: 1999 either is or is not what was meant, with nothing in between for a rounding step to introduce.

5.4 Cross-currency arithmetic is refused, not coerced

A `money` value has one currency. Summing two `money` values with different currency codes requires a conversion rate, and a rate is a claim about a moment in time that neither party necessarily agreed was authoritative — unlike a price, which the seller signed directly. Converting one side and adding therefore produces a total that looks exact and is not one either party actually committed to. TRACT's position is that this arithmetic is refused rather than performed: a sum is only defined across values sharing one currency, and this is why an `Order`'s `total` (§16.6) is a single `money`, consistent with an order being scoped to one seller (§16.6, "one order per seller is a grammar-level property") who is expected to quote consistently.

The grammar does not yet enforce this **within** a single `Consideration` value, and that is a real gap rather than a settled point: `DepositBalance`'s two `money` fields, and each `PriceTier`'s `unit_price`, each carry their own independent currency code. Nothing today rejects a deposit denominated in one currency against a balance denominated in another, even though the two are instalments of one price rather than two independent prices.

5.5 Tiered / volume pricing

`[+ PriceTier]` — at least one tier, each a `(min_qty, unit_price)` pair. The natural selection rule is the tier with the greatest `min_qty` not exceeding the requested quantity, standard volume-pricing practice. Two things the grammar does not currently pin down:

- **Ordering and duplicates.** `ProductRecord`'s attribute list is specified as sorted and deduplicated (§16.5.1); the `PriceTier` array carries no equivalent canonicalisation rule, so two tiers naming the same `min_qty` are not excluded and their relative order is not defined.
- **Coverage below the lowest tier.** `[+ PriceTier]` guarantees at least one entry, not one whose `min_qty` is 1 — a quantity below every published tier's threshold has no defined price.

5.6 Recurring — reusing §3's calendar machinery

`recurring` carries an amount and an RFC 5545 `RRULE`, the same standard §3.4 profiles for availability, and for the same reason: a seller who already runs billing or calendar software against RFC 5545 recurrence should not need a second schedule format to describe a subscription period. The amount is fixed per period — a recurring charge that also varies by usage is `metered` (§5.7), not `recurring`, and a trade needing both is two axes' worth of behaviour that this grammar does not currently let one `Consideration` value express at once.

What is unresolved is how a recurring consideration interacts with §18.3's order state machine, which runs one lifecycle from `draft` to a terminal state and does not loop. Whether each renewal period is a fresh `Order` — reasonable, since `Order` is a lightweight sealed object and "was this specific charge accepted" is naturally per-period — or one `Order` that somehow persists across periods is not decided, and §18.3 as written does not describe a renewal path either way.

5.7 Metered

`metered` carries a dimension string (calls, kWh, gigabytes) and a unit price, billed after consumption is known. Two consequences the grammar does not yet resolve:

- **No usage-attestation object exists.** §16.6 defines no object for reporting or disputing how much of the metered dimension was actually consumed, so the mechanism by which a metered charge's final figure gets attested by one party and verified — or contested — by the other is unspecified.
- **`Order.total` is mandatory, and a metered order's total is not knowable up front.** §16.6 defines `total` as key 4 with no `?` — it is required on every `Order`, placed at `draft` / `placed` time. A metered offer's actual charge is only known after fulfilment. Whether `total` for a metered order is an estimate, a cap, or whether metered consideration needs a different order shape entirely is open (§5.13).

5.8 Deposit + balance

Deposit payable at order, balance payable at delivery or completion — the shape a genuine deposit-taking trade has, rather than modelling it as two unrelated fixed prices. It pairs naturally with `ReturnRequired` fulfilment (§4.7): the deposit is the mechanism that makes a late-return or damage claim on a rental whole, and §4.7 is explicit that condition and lateness are handled as an order-level dispute, not as a fulfilment-axis field. §5.4 already flags that the grammar does not require `deposit` and `balance` to share a currency, which is the sharpest instance of that gap.

5.9 Quote-required — RFQ and B2B contract pricing

`quote-required` carries nothing (`{ 8 => null }`): the published offer states only that no price is published, and that a buyer must ask.

The mechanism does not need a new object type for the answer. A seller's response to a quote request is an ordinary `Offer` — `Fixed`, `Tiered`, whatever consideration the seller actually wants to charge this buyer — distributed differently from the rest of the catalogue: handed directly to the requester's key rather than published in the open feed. This is exactly how B2B contract pricing works under the same mechanism: a

per-buyer price is nothing more than an `Offer` addressed to one key instead of broadcast to all of them. No pricing-tier object, no buyer-group primitive — the existing `Offer` grammar already expresses "this price, for you specifically" the moment its distribution, not its shape, is restricted.

Two things this leaves unspecified:

- **The request itself has no defined wire shape.** A buyer asking for a quote is plausibly sealed — it may disclose volume or identity a seller would rather not publish an answer about openly — but §16.6 defines no such object today.
- **Nothing links a private per-buyer `Offer` back to the request that produced it.** Without that link, a buyer holding such an offer has no signed record of having asked for it, only of having received it.

5.10 Tax treatment categories belong here; tax rates do not

The offer carries a **treatment category** — standard-rated, zero-rated, exempt, reduced, and whatever taxonomy a given regime uses — plus the anchors §11.2 derives, principally the place of supply this axis's counterpart, §4, computes. It does not carry a **rate**. Rates change by jurisdiction and by week; encoding one into a specification guarantees the specification ships stale law the day a legislature changes a number. Rate lookup, keyed by treatment category, place of supply, and the applicable date, is deliberately left to whatever source an implementation trusts — a jurisdiction's published schedule, a commercial rate service — never to a table in this document.

Neither `Consideration` nor `Offer` currently has a field for the treatment category itself (§16.5.2) — this section states the policy the grammar has not yet been given a slot to carry.

5.11 Currency conversion is a presentation concern

An offer's `money` is denominated in one currency, and that is the currency a resulting `Order`'s `total` and any `PaymentAttestation` (§16.6) are bound to — the **settlement currency**. A storefront or client may show a buyer a converted estimate in a currency they prefer while browsing, computed live against whatever FX source that display layer trusts, and visibly marked as an estimate rather than a price. That converted figure never becomes what gets signed; §5.4 and §16.7 are the same rule applied at two different points — refused in arithmetic, estimated only in presentation.

5.12 Standards profiled

ISO 4217 currency codes. RFC 5545 recurrence rules, reusing §3's machinery rather than inventing a second schedule format.

5.13 Open

- Whether `Consideration` values with more than one `money` field (`deposit + balance`, and each `PriceTier`'s `unit_price` against its neighbours) need a grammar-level rule requiring a shared currency (§5.4, §5.8).
- Whether `PriceTier` needs a canonical ordering rule and duplicate-`min_qty` rejection the way `ProductRecord`'s attributes have, and what a quantity below every published tier resolves to (§5.5).
- How a `recurring` consideration's renewal periods map onto §18.3's single-shot order lifecycle — fresh `Order` per period, or some other shape the state machine does not currently describe (§5.6).
- Whether `metered` consideration needs a usage-attestation object, and how a metered order satisfies `Order.total` as a mandatory field when the true amount is only known after consumption (§5.7).
- Whether a quote request needs a defined sealed wire shape, and whether a private per-buyer `Offer` needs a field linking it back to the request that produced it (§5.9).

- Where the tax treatment category is actually carried on the wire, given neither `Consideration` nor `Offer` has a field for it today (§5.10).

6. Cart

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

6.1 Scope

Buyer-side cart state, live availability, and reservation semantics without a central lock.

6.2 What this section will specify

- The cart as **buyer-held CRDT state**, synced across the buyer's own devices via substrate capability ③. No seller and no gateway holds it.
- Live re-evaluation: subscribing to availability feeds for items already in the cart.
- **Reservation and hold** semantics, including what a seller may promise and for how long.
- **Bounded-counter inventory** across a seller's own replicas: stock partitioned into per-replica quotas, sold freely within quota, quota transferred between replicas on demand. The invariant is that total sales never exceed total stock, without a coordinator.

6.2a What the bounded counter actually costs (§21.10)

The mechanism is real and shipped: the escrow/demarcation family (O'Neil 1986 → Barbará-Millá & Garcia-Molina 1994 → Balegas et al., SRDS 2015), in production as AntidoteDB's `antidote_crdt_counter_b`. Safety holds under arbitrary message loss, delay, reorder and partition, because the locally computed right count is conservative — it can under-count but never over-count. For contrast, a plain read-check-decrement counter measurably *does* oversell: about 200 excess decrements at ~200 concurrent clients in the published experiment.

What §21.10 surfaced is that **four operator-shaped roles hide inside it**, and this section has to accept or replace each rather than let "no coordinator" read as "no coordination anywhere":

1. **An intra-replica serialization point is mandatory** — a single-writer node, a compare-and-swap, or consensus. A replica therefore cannot itself be a set of mutually uncoordinated nodes. The coordination is not removed; it is pushed down one level and made local.
2. **The failure model is crash-recovery with durable state, not Byzantine.** This is the hard boundary. A lying or state-losing replica simply oversells, and the literature covers **a seller's own replicas, which trust each other** — not stock held across mutually distrusting parties. Bounded counters protect a seller from their own concurrency; they do **not** protect a buyer from a dishonest seller, and conflating the two would be a safety claim nothing supports.
3. **Reclaiming a dead replica's rights is a fallible decision.** If the failure detector is wrong, rights are double-issued and the invariant breaks. Deciding a peer is permanently gone is exactly the judgement a coordinator makes, so it needs a stated failure mode rather than a background sweep.
4. **At three or more replicas, transfer names a recipient.** It is no longer anonymous, which reintroduces an allocation policy — and the source papers suggest centralising it. This section has to say who chooses.

6.3 Prior art

Escrow-based / bounded CRDT counters from the distributed-systems literature. Saga and compensating-transaction patterns for the multi-party case.

6.4 Honest limits this section must state

- A partitioned replica holding unused quota **strands** that stock until it rejoins.
- There is **no cross-seller atomicity**. A multi-seller cart is a set of independent orders; one seller declining does not roll back another's acceptance. Checkout is compensating actions, never a distributed transaction — that would need a coordinator with authority over sovereign parties. Interfaces must show per-seller status rather than a single "order placed".

6.5 Open

- Quota rebalancing policy: how aggressively replicas should transfer, and whether the spec should mandate one or leave it to implementations.

7. Order

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

7.1 Scope

The sealed order object, its lifecycle, and the evidence each transition produces.

7.2 One seller, one sealed order

`Order` is a sealed message (§16.6): encrypted, per-party, never published, held at the two endpoints where deletion is meaningful (§0.5). It names exactly one seller and carries only that seller's lines — not by convention but by the shape of the object itself: there is no field a cross-seller order could occupy (§16.6). A buyer's cart (§6) is the only place a multi-seller view exists; checkout is where it stops existing anywhere else. A cart spanning five sellers produces five independent sealed orders, one per seller, and none of the five carries any trace that the other four exist — not their sellers, not their lines, not their total. Each seller learns only what §16.6 gives it: its own lines, and this buyer's total for those lines.

7.3 Who signs, and what it proves

§18.3 has the full machine — every transition, its trigger, its timeout. This section does not keep a second copy of that table, because a fact stated twice is a fact that can disagree with itself the day only one copy gets edited (§19.1). What it adds instead is the part §18.3's table doesn't carry: what each signature is evidence *of*, and — for the one transition that genuinely varies — what "delivered" actually looks like.

Two of the transitions carry evidence that is straightforward to state once. `placed` is the buyer's own commitment to specific lines, a specific seller and a specific total, nothing more. `accepted` / `declined` / `countered` is the seller's own response, signed rather than inferred, which is precisely why §18.3 treats silence as decline rather than acceptance — an inferred "yes" would be a signature that was never made.

`delivered` is not one kind of evidence — it is whichever the Fulfilment axis (§4) says it is. §18.4 states that handover evidence exists for a `fulfilling → delivered` transition; it does not say what that evidence is, because the answer depends on which Fulfilment variant the line was placed against, and §4 is where that mapping belongs. A shipped line closes on a carrier's proof-of-delivery, signed by the party taking custody (§18.4) — not the sender. A digital-grant line closes the moment the grant is issued, signed by the seller alone, with no carrier anywhere in it. A perform-at-place line closes on attendance at the stated time and place. An `Order` can carry lines under different variants at once, and `Order.state` is one field for the whole message (§16.6): it only reaches `delivered` once every line's own evidence exists, which for a mixed order can mean waiting on the slowest one.

`closed` is the one edge that can carry no signature at all. `CONFIRM_TIMEOUT` expiring into `closed` (§18.3, §19.2) is deliberate, not an oversight — §18.3 states why: a buyer who stops responding must not be able to strand a seller's money by doing nothing. But the evidence behind that specific transition is elapsed time plus whatever was last signed, not a buyer's word, and this section should not let anyone read the time-out path as an implicit confirmation.

7.4 Amendment

Whether an amendment is a new sealed `Order` superseding the one it changes, or an entry in an append-only operation log against the original, is open at the wire-format level (§16.8), and this section inherits the same open question rather than quietly picking a side. The two options pull toward different parts of the design that already exist elsewhere in this specification. Supersession is the pattern the rest of the document already uses — an offer's withdrawal is a successor object, not an edit (§18.2); a review's retraction is a superseding tombstone (§10.2) — so amendment-by-supersession would be one mechanism doing a job this specification asks for repeatedly, rather than a special case invented for orders. A log is easier to audit for a different reason: the full history of what changed and when is a linear read, where a chain of superseding objects has to be walked backward through content addresses to reconstruct the same story. Neither is decided here; §16.8 is where the choice, once made, belongs.

7.5 Partial fulfilment

An order can be split across time even when it cannot be split across sellers: a line back-ordered while the rest ship, or a quantity short-shipped against what was accepted. The wire format as it stands has nowhere to put that — `Order` carries one `OrderState` for the whole message (§16.6), and `OrderLine` carries no status of its own. Two shapes could close that gap: a per-line status field added to `OrderLine`, or partial fulfilment expressed as an amendment (§7.4) that splits one order into a closed successor for what shipped and a still-fulfilling successor for what didn't. Given §16.8's lean toward supersession, the split reads as the more consistent shape, but nothing here commits to it — this is the kind of decision that has to be made once, at the grammar, rather than differently by every implementation.

7.6 Partial refund

This one has less distance to travel. `PaymentAttestation` (§16.6) already carries a payer, a payee, an order reference and an amount — a partial refund is a second attestation against the same order, payer and payee reversed, amount set to the difference. Nothing new needs adding to the grammar for the object itself; what remains open is only whether the order's own state machine (§18.3) needs a `refunded` or `partially-refunded` marker, or whether the payment-attestation trail is sufficient evidence on its own, with nothing said about it at the order level.

7.7 Returns and exchanges

A return only has a shape where the Fulfilment axis (§4) gave it a place to go back to. `collect` and `perform-at-place` have nothing to reverse in the courier sense — undoing them is a refund or a re-performance, not a shipment. `digital-grant` and `access-grant` don't have a return leg either; "return" there means revoking a grant already issued, which is a licence-terms question for §4.5 and §5, not a delivery question for this section. Two variants carry an actual carriage leg back:

- **`ship`** — the goods have to travel back to the seller, and nothing in Incoterms 2020 (already profiled for the forward leg, §4.4, §8.3) speaks to who pays for the reverse one. Who bears return carriage — buyer, seller, or split by reason for return — is a term this section has to give a home to, and it isn't decided yet whether that home is a default on the offer, a field on the order, or left entirely to the parties to negotiate.
- **`return-required`** (§16.5.2) — the Fulfilment variant already names a place and a term (§4.2, §4.3), because the whole point of that variant is that the goods were always going back. Who pays to get them there is the same open question as `ship`'s, with the same shape.

Several consumer-protection regimes assign return-carriage cost by rule for a change-of-mind return inside a cooling-off window (§11.4 lists which regimes this section has to accommodate), which argues for letting a regime-mandated term override whatever an offer declares by default rather than the other way round. Whether that override is expressed here or belongs entirely to §11 is unresolved.

7.8 One seller's decline is not another's rollback

A cart spanning several sellers produces several independent orders (§7.2), and one of them declining, timing out, or being cancelled has no effect on the others' state. This is stated as a property, not a limitation apologised for: a distributed transaction across sovereign parties needs a coordinator with authority over all of them, and that coordinator is precisely the thing this design does not have and is not trying to grow. §6.4 already states the consequence for the cart; it is repeated here because it is the order machine, not the cart, that a reader might otherwise expect to paper over it with a single "order placed" status.

The literature backs this up directly rather than leaving it as an assertion: sagas — the standard pattern for exactly this multi-party situation — provide **neither atomicity nor isolation** (§21.10.3). Other parties can observe a partially-completed checkout, and there is no abort cascade that unwinds an accepted order because a different seller declined. An interface built on top of this section has to show per-seller status, because a single aggregate status would be reporting a guarantee the protocol does not make.

7.9 Standards profiled

The substrate's sealed message object for transport. Where an order must be exported to a legacy counterparty (an ERP, a 3PL), the mapping targets are UN/CEFACT and EDIFACT order/despatch/invoice messages; that mapping is informative, not required.

7.10 Open

- Whether order amendment (§7.4) is a superseding sealed object or an operation log — supersession fits the pattern the rest of the document uses; a log is easier to audit. Undecided at the wire-format level (§16.8).
- Whether partial fulfilment (§7.5) needs a per-line status field on `OrderLine`, or is represented as an amendment that splits one order into two. The grammar has no per-line status today either way.
- Who bears return carriage on a `ship` or `return-required` line by default (§7.7), and whether a regime-mandated term is allowed to override it.

8. Delivery

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

8.1 Scope

Rate cards, legs, consignments, consolidation, and local route computation.

8.2 `RateCard`: published, not quoted

A `RateCard` (§16.5.3) is a signed public object: a zone table, weight brackets, a dimensional divisor, surcharges, served countries, excluded categories, and a `published` timestamp. A leg's price is **computed by the buyer's node from a locally held copy**, not returned by a call to the carrier. That is a substitution of architecture, not just of API, and it has consequences beyond convenience:

A quote API implies	Publishing removes
a rate limit, so the carrier can throttle callers	none — the buyer's node reads its own copy
an API key, so the carrier knows who is asking	no key; the object is public and unauthenticated to read
a cost per quote, since carriers meter API calls	zero marginal cost once the card is fetched
the carrier — or whoever operates the quoting endpoint — learning what is being bought, by whom, in what quantity, and how often	nobody outside the buyer's own device runs the computation

The object type carries no privilege tier. A `RateCard` is keyed to whoever published it — a national carrier or a neighbour with a bicycle, identically typed (§0.4.1's courier and distributor rows) — so the grammar cannot express a "professional" rate card distinct from an amateur one. A route comparison that mixes a multinational's zone table with a peer courier's flat local rate is one computation over two instances of the same type, not two code paths.

Cards drift: surcharges change, zones get renumbered. `published` exists so a stale card is not replayed silently, and `RATE_CARD_MAX_AGE` (30 days, §19.6) is the point past which a locally computed price is stale enough to mislead — the buyer is told the figure is old rather than shown a confident number computed from it.

8.3 Volumetric weight and the divisor floor

`billable = max(actual, L·W·H / dim_divisor)`. Every major carrier prices this way, because a router that used actual weight alone would under-quote large, light parcels — the buyer would discover the shortfall at the counter, which is exactly the surprise this design exists to remove by computing the number before checkout rather than after.

`dim_divisor` is the field a hostile or careless publisher can use to distort every quote computed from their card: the smaller the divisor, the larger the volumetric figure, and volumetric weight already dominates actual weight for anything bulky and light. `MIN_DIM_DIVISOR` is 1000 (§19.5). Real carriers publish 5000 or 6000; nothing between 1 and 1000 corresponds to a divisor any carrier actually uses, so a card in that range is treated as unusable rather than as an aggressive but legitimate rate. Zero is the one exception and means

something different — “this carrier does not apply volumetric weight at all” — not an unset field, so it is accepted rather than rejected.

`dim_divisor`	Meaning	Accepted
0	carrier does not apply volumetric weight	yes
1–999	not a divisor any carrier publishes; inflates every parcel priced against the card	no
≥ 1000	ordinary range; 5000 and 6000 are the common real-world values	yes

This is checked before the card is used to price anything, because a rate card is a claim by its publisher and arrives from a stranger — it is validated, not trusted, the same way any other public object arriving unsolicited is (§14.3).

8.4 Legs, consignments, and custody

A **leg** (§0.9 glossary) is one movement of goods between two places, priced by one rate card. A **consignment** is physical goods in someone else’s custody — a courier leg or a distributor hold — and moving through several legs means moving through several consignments in sequence, each with its own custodian.

Custody changing hands is attested by a **signed custody handoff**, and the full lifecycle — `created → accepted → in-custody → handed-off → delivered`, who signs each transition, and what each timeout expires into — is specified once, in §18.4, and is not repeated here. The one property worth restating because it shapes everything else in this section: a handoff is signed by the party **taking** custody, not the party giving it up, because a chain attested only by senders proves someone tried to hand something over, and a chain attested by receivers proves the goods actually moved.

8.5 Consolidation: three candidate routings

A cart with lines from several sellers can reach the buyer by more than one shape, and the three worth comparing exhaust the useful design space:

Shape	Route	Wins when
Direct	every seller ships straight to the buyer	speed per item matters more than total cost
Hub-near-buyer	sellers ship to a distributor near the buyer, who combines and sends one parcel	the last mile dominates cost; the buyer accepts waiting for the slowest seller
Hub-near-sellers	sellers who are geographically close consolidate first, then one long leg carries the combined parcel	the long haul dominates cost

Each candidate is scored the same way:

```
total = Σ(leg costs) + storage_per_day × wait_days + handling_fee
```

`storage_per_day`, `handling_fee`, and `wait_days` come from the hub’s `CapacityRecord` (§16.5.3) when a hub is involved, and are zero for `Direct`. Comparing three fully-priced totals is the whole routing decision.

No optimiser is specified or needed. The candidate set is small by construction — a handful of hub choices, not a combinatorial search over carriers and paths — so a buyer’s own device evaluates all three exhaustively and picks the lowest total. A specification that reached for a solver here would be adding machinery to a problem that does not have enough variables to need one.

8.6 Distributors: the consolidation role

A **distributor** publishes a `CapacityRecord` (§16.5.3) — country, coarse locality, storage per item per day, a handling fee, available slots, and excluded categories — and holds goods in transit on that basis. Entry is permissionless: a keypair and space is the whole requirement, the same bar as every other role in §0.4.1.

Economically this is the freight-forwarder pattern that already runs worldwide. What differs is that the terms are a signed object anyone can read and compute against, rather than a bilaterally negotiated contract a buyer's node has no way to see. A distributor competes on the same published terms a carrier competes on rate cards, and switching between them costs a routing recomputation, not a renegotiation.

8.7 Honest limits this section must state

- **Redistribution of negotiated rates may be restricted.** Some carriers' API terms forbid republishing negotiated rates. Published list rates, and a seller's own rates they choose to publish, are unaffected; republishing a carrier's confidential negotiated rate is the *publisher's* compliance problem, not the protocol's. An offer may declare that a live quote is required (§16.5.3) instead of pointing at a card, precisely for this case.
- **Transit figures are estimates, not commitments.** `transit_days` on a `Zone` (§16.5.3) is the carrier's own claim, and `TRANSIT_TIMEOUT` (carrier estimate × 3, §19.3) is built around that being unreliable rather than around it being wrong by a fixed margin.
- **`wait_days` is the least reliable input in the whole computation.** It depends on the slowest seller in a consolidation actually dispatching on time, which is outside any rate card's or distributor's control, and it must be presented to a buyer as an estimate rather than folded into a total that reads as a promise.
- **Custody attestations prove transfer, not recoverability.** A signed handoff establishes who held the goods and when. It establishes nothing about getting them back. `lost` is a terminal state in §18.4 with a signed history attached and no remedy attached — where the goods went is answerable; being made whole for them is §9's problem, or nobody's.
- **This section is unevidenced.** Three research passes over the literature returned no verified findings on logistics standards, carrier API terms, or consolidation optimisation (§21.1); the gap is recorded there as absence of evidence, not evidence of absence. Everything above is design reasoning checked for internal consistency, not a claim checked against how carriers, distributors and couriers actually behave, and it should be read with that confidence level and no more.

8.8 Open

- Whether a peer courier needs a distinct rate-card shape — flat price per kilometre — or should be expressed inside the same zone-table model as a national carrier, at the cost of a zone table with one entry.
- Whether `RATE_CARD_MAX_AGE` should differ between a `RateCard` and a `CapacityRecord`. Zone tables and storage rates plausibly drift at different speeds, and today one parameter (§19.6) covers both.
- Whether consolidation route choice needs to be reproducible across independent implementations given the same inputs, or whether two conformant buyer nodes may legitimately land on different totals because their copies of the same rate card are at different points within `RATE_CARD_MAX_AGE`.

9. Settlement

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

9.1 Scope

The payment seam, rail classes, escrow, and the gateway's money role.

9.2 The seam names no provider

TRACT specifies where money crosses the boundary and what must be verified. It specifies no rail, currency, token or ledger. Naming a provider in a protocol exports that provider's jurisdiction and licensing to every implementor.

9.3 Rail class is part of the type

`CustodialReversible` (chargebacks exist; the card network is already the adjudicator) versus `NonCustodialFinal` (nobody custodies, nothing reverses). An implementation must not substitute one class for the other without an explicit party decision, because the buyer's recourse differs.

9.4 What this section will specify

- The payment-attestation object: payer, payee, order, amount, rail class, external settlement reference. **The protocol carries attestations, never funds.**
- Escrow lifecycle: fund → hold → release / refund / split, each a signed object.
- `EscrowScope`: buyer countries, seller countries, supply countries, currencies, rail classes, value ceiling, excluded categories, claimed authorisations.
- **Fail-closed scope intersection** at checkout, and the requirement that an empty intersection is disclosed rather than silently downgraded.

9.5 Escrow is the operator class

It requires legal standing, a payment-provider relationship, a float, and licensing — none of it derivable from a keypair. What bounds it: permissionless entry, competition, per-order choice by both parties, no access to identity keys, and **every ruling published as a signed object**, so an operator that rules unfairly accumulates a permanent verifiable record.

9.5a The measured cost of optionality (§21.6)

§9.5 keeps escrow per-order and optional, and §9.6 gives the reason: mandatory escrow would exclude regions no licensed operator serves. That reasoning is sound and the consequence is still real.

OpenBazaar's escrow was also opt-in, and **bad actors simply declined it** — the protection went unused precisely where it mattered most. Both cannot be true for free, and this specification pays on the second. What follows from that, rather than pretending it away:

- an unescrowed trade is an explicit, disclosed outcome shown to both parties before they commit, never a silent default;
- interfaces render the absence of escrow as a fact about the trade, not as a missing field;
- a seller declining every escrow provider a buyer trusts is itself information the buyer should be given, since it is the observable form of the failure above.

9.6 Honest limits this section must state

- Physical custody cannot be made trustless.
- Non-custodial programmatic escrow (multi-sig, hashlock+timelock) removes the custodian but **deadlocks on genuine disputes** — the exact case it was wanted for.

- Unescrowed trade must remain possible, or scope mismatches would exclude underserved regions.

9.7 Open

- Whether partial release (split rulings) needs protocol representation or is an operator concern.

10. Trust

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

10.1 Scope

Reviews, purchase attestation, and ranking — without creating an authority.

10.2 What this section will specify

- `Review` as a signed public object attached to a product address, seller, distributor or courier.
- **Purchase attestation**: a proof, issued by the seller or an escrow operator at completion, that the author actually transacted. Verifiable by anyone, unlike a platform's "verified purchase" badge, and it makes ballot-stuffing cost a real transaction.
- Per-seller pseudonymous authorship (§1.2).
- Seller responses, and supersede-based retraction.

10.3 The prohibition

No network-wide published score. Computing one requires a party that aggregates and ranks, which is the authority the design removes. Ranking is derived data; indexes will differ, and that is the intended outcome, not a defect.

10.3a What attestation does not fix (§21.6)

§21.9 obliges this section to state the measured outcome rather than the hypothetical one.

OpenBazaar's reputation failure was self-published, unweighted reviews with no banning authority, and **one vendor faked 60% of measured sales value**. A purchase attestation is strictly stronger than what OpenBazaar had — it binds a review to a real transaction, so ballot-stuffing costs actual trades. Two of the failure conditions nonetheless survive unchanged:

1. **Self-dealing produces genuine attestations.** A seller transacting with itself generates real proofs. Attestation raises the cost and leaves a public trail; it does not establish that the counterparty was independent, and nothing in this document does.
2. **Escrow-issued attestations are scarcest where they would matter most**, because escrow is opt-in and declined by exactly the actors it constrains (§9.5a).

The achievable Sybil-cost floor on a signed-feed substrate is an **open question** (§21.8), not a solved one. An index's weighting policy is where that judgement lives, and different indexes reaching different answers is the design rather than a defect.

10.4 Honest limits this section must state

- Rankings disagree between indexes; buyers lose the convenience of one authoritative number.
- Attestation raises manipulation cost but does not eliminate it — a seller can transact with itself, expensively and visibly.
- Whitewashing is bounded, not solved: a new key has no history, and discounting new keys is the only available defence.
- **Erasure**: a review is public, irrevocable, and signed by a person. Retraction is a superseding tombstone honoured by conformant clients and gateways; the residual — bytes persist if any independent holder keeps them — must be disclosed to the author before publishing.

10.5 Prior art

Sybil, whitewashing and ballot-stuffing literature on decentralized reputation; web-of-trust and locally-measured reputation as the alternatives to a global score.

11. Jurisdiction

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

11.1 Scope

Making the legally responsible party explicit, and getting tax anchors right by construction.

11.2 The four anchors

The most common commerce-tax error is conflating party location with where a supply happens. These are four distinct fields, derived from four different places:

Anchor	Derived from	Governs
seller establishment	seller identity	licensing, seller-side registration
buyer residence	buyer disclosure at order	consumer-protection rights (generally non-waivable)
place of supply	**the Fulfilment axis (§4)**	VAT/GST, especially services and events
delivery destination	the shipping leg (§8)	customs, duty, product-safety regimes

The forcing example: an event held in one country, sold by a seller in another, to a buyer in a third. Admission to events is generally taxed where the event physically takes place, so only the Fulfilment object knows the answer.

11.2a What the law actually says, as far as anyone has checked (§21.11)

This section previously answered its central question by assertion. A narrow legal pass has now checked one of the five questions it rests on, and the answer is **structurally favourable, more limited than assumed, and entirely untested**. The other four remain unresearched after three passes — including the GDPR erasure conflict, which is still the most likely hard blocker.

The argument that survives. Every representative US state facilitator test is *conjunctive and gated on a contract with the seller*: Cal. Rev. & Tax. Code §6041(b) — "a person who **contracts with** marketplace sellers …

and who does both of the following" — puts even its infrastructure prong inside that gate. A gateway no seller has contracted with is outside the definition on its face, whatever it runs. No court has ever tested this.

Three corrections this section has to absorb.

There is a marketplace. The term is medium-agnostic and expressly enumerates a catalog and a dedicated sales software application, so a signed catalogue feed and a buyer-side cart client are within it. The available argument is that there is **no facilitator** — never that there is no marketplace.

Escrow is the trigger, and in two states it is enough alone. Tex. Tax Code §151.0242(a)(2) catches "directly or indirectly processes sales or payments" with no carve-out; N.Y. Tax Law §1101(e)(1)(B) catches a person who "contracts with a third party to collect" receipts, so routing checkout through the gateway's own provider suffices even if its balance sheet never holds the money. §9's "escrow is an operator class" understates this: escrow is also what most likely makes that operator a **tax** facilitator.

Render-only is not a safe posture. It holds in New York and Texas and is likely caught in Washington and California through the listing and order-taking prongs. Two states of roughly fifty is not a US position.

One thing is confirmed: a self-hosted seller taking direct payment for their own goods is never a facilitator, because every definition requires sales by persons **other than** the operator of the medium.

11.2b The EU rule written to defeat this argument

Art 5b of Council Implementing Regulation (EU) 282/2011 treats an interface as not facilitating only if, cumulatively, it sets **none** of the terms directly or indirectly, is **not** involved in authorising the charge, and is **not** involved in ordering or delivery. Carrying out even one may suffice to make it a deemed supplier.

The Commission's Explanatory Notes then name this design's central claim and reject it: the indication "that the contract is concluded between the underlying supplier and the customer **is not sufficient**" to escape deemed-supplier status, because the concept "goes beyond the contractual relationship and looks at the **economic reality** and in particular the **influence** exercised". The words "indirectly" and "any" are stated to exist precisely to prevent "artificial splitting of rights and obligations between the electronic interface and the underlying suppliers".

"The contract is between two keypairs" is the argument that text anticipates. A protocol asserting a contractual arrangement does not escape a test that measures influence rather than declarations.

The scope limit, given plainly rather than as reassurance: Art 14a bites on imported consignments of intrinsic value ≤ €150 and on intra-EU supplies by non-EU-established sellers. It is not a general rule for all EU commerce. Where it applies, no protocol design argues its way out, and this section does not pretend one could.

11.3 What this section will specify

- **Responsibility follows the money**: every order names seller of record, facilitator (the gateway, if it settled), importer of record, in-region responsible person, and escrow/rail.
- Geo-availability on offers, and fail-closed construction when a required in-region role is absent.
- Tax treatment categories (not rates — see §5.5).

11.4 Regimes this section must accommodate

South Africa (electronic-transaction disclosure and cooling-off, consumer protection, POPIA, VAT, payment-side KYC); the EU (GDPR, platform trader traceability, consumer rights, in-region responsible person for product safety, VAT one-stop schemes, platform reporting); other African markets (national data-protection acts, local VAT registration, regional trade frameworks); New Zealand (privacy, fair trading, consumer guar-

antees, GST on low-value imports); the Americas (US economic nexus and marketplace-facilitator rules, seller-traceability legislation, Canadian and Brazilian privacy law).

The protocol guarantees the **facts** a regulator asks for are present, signed and attributable. It does not make any deployment compliant, and must not be read as legal advice.

11.5 The erasure conflict, resolved structurally

Published objects are irrevocable; erasure rights cannot be satisfied against them. Therefore **no personal data enters the public quadrant** (§0.5.1). Orders and everything identifying a person are sealed and deletable at the edges. Reviews are the single bounded exception (§10.4).

12. Gateway

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

12.1 Scope

The storefront role: rendering signed objects for browsers, domains, and the trust boundary.

12.2 What this section will specify

- The rendering contract: what a gateway must verify before serving, and what it publishes about what it served.
- **Render bundles**: seller-supplied presentation, sandboxed.
- **Subdomain** allocation (single-label, one wildcard certificate) and **custom domain** binding.
- Escrow co-location: a gateway that also settles is the facilitator of §11.3.

12.3 Origin isolation (will be normative)

Merchant render bundles are untrusted code. **Every store must get its own origin.** A gateway serving multiple stores from one origin lets one bundle read another store's cart and session; it is non-conformant, not merely ill-advised.

And this section has to say how origins are compared, which it previously did not. Implementing the rule surfaced the gap: a naive string comparison reports `alice.example` and `Alice.example` as distinct origins, while DNS and every browser treat them as the same one — same storage partition, same cart, same session. A gateway that allocated both would believe it had isolated two stores and would in fact have handed one merchant's bundle read access to the other's data, while passing its own conformance check.

So the comparison is normalised before it is made: ASCII-lowercased, with any trailing root label dot removed. Internationalised names need an IDNA pass before that point, and a gateway accepting them without one has the same hole in a less obvious form. A rule that says *what* must be true and not *how it is tested* is a rule an implementation can satisfy on paper and violate in practice.

12.4 Process isolation (will be normative)

A gateway terminates untrusted connections and renders untrusted bundles. It must run as a separate process with no access to identity keys or the object store. "One binary, several roles" never means one ad-

dress space.

12.5 The honest limit

DMTAP's gateway self-extinguishes as adoption grows. **TRACT's does not**, because browsers are permanent and a shopper without a keypair cannot verify a signature. The mitigation is **detectability, not prevention**: any node can re-render the same store from the same signed objects and be compared byte-for-byte. A native client needs no gateway; two native parties never need one to transact.

12.6 Open

- Whether the spec should define a canonical render so byte-for-byte comparison is meaningful, or only require that the underlying objects be exposed for independent verification.

13. Analytics

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

13.1 Scope

Measurement for sellers that does not require per-visitor surveillance: what a seller may learn about traffic to their store, and by what mechanism, without a hosted analytics platform sitting between them and every visitor.

13.2 Posture

Tiered and buyer-granted, with an aggregate floor. Four stages, each with a different default — and the default is the point, since a merchant cannot be handed more than the stage it sits in discloses:

Stage	Default	Why
Browse	anonymous	catalogue reads are pulls of public objects (§2); a fetch against a signed feed carries no session for an identity to attach to
Grant	opt-in, scoped, revocable, expiring	the buyer's node decides what to disclose and to which store; a seller cannot be handed more than the grant states
Order	full detail	the seller needs an address to fulfil (§4), and the buyer knowingly provides it as part of placing the order (§7)
Aggregate	counts, with no visitor-level record	folded before it leaves the point of collection; the object type has no field capable of holding one (§13.4)

Both grants (§13.3) and orders (§7) are sealed, never published (§0.5): a disclosure a buyer chooses to make to one seller is not a public object any node can read, and it is not the seller's to republish.

13.3 The grant object

A grant is a small signed object a buyer's node sends directly to one seller — scoped to a single store, carrying an enumerated field list, and time-bounded — rather than a blanket permission the seller then interprets.

Field	Carries	Note
`store`	the seller's public key the grant is scoped to	one store per grant; a grant issued to one seller discloses nothing to another — there is no field meaning "all stores"
`subject`	a per-store pseudonymous key, distinct from the buyer's persistent identity	reused across visits to *this* store only, so a seller can recognise a returning visitor without the grant doubling as a cross-store identifier (§13.6)
`fields`	which of {coarse geography, referrer, session-continuity} the buyer discloses	enumerated; a field absent from the list is not disclosed, not disclosed-as-empty
`issued_at` / `expires_at`	timestamps	defaults to `GRANT_LIFETIME` (24 hours, §19.6) unless the buyer's node states a longer value explicitly
`signature`	over the above, by the subject key	proves the grant came from whoever holds that pseudonymous key, not from a seller recording consent after the fact

Expiry is not a courtesy. A grant with no expiry is a permission that can only be withdrawn by an affirmative act, and a buyer who forgets, moves on, or replaces the device that issued it never performs that act. `GRANT_LIFETIME` makes silence the buyer's default rather than the seller's — the same asymmetry §18.3 states for order timeouts, applied to consent rather than money: a grant that never expires is consent that cannot be withdrawn by inaction.

Revocation is a further signed object naming the grant it withdraws, sent the same way. A seller that receives one stops relying on the grant from that point forward; it does not, and structurally cannot, unwind an aggregate count the grant already contributed to before revocation (§13.8).

13.4 The aggregate telemetry object

What a seller's own node folds its request log and its received grants into before treating the result as a number to look at, rather than a record to query per visitor. This object stays on the seller's own node; nothing here specifies publishing it anywhere.

Field	Carries
`window`	a bucketed time period (hour or day) — never a timestamp finer than the bucket
`stage_counts`	funnel-stage counts (viewed / carted / ordered) for the window
`geography`	counts by country or region (ISO 3166-1), never finer than the seller declares
`referrer_class`	counts by coarse referrer category, not a full referrer URL

There is no visitor-identifier field in this object's definition — not one left empty, one that does not exist. A field that is merely unpopulated can be filled in later by a different code path; a field absent from the type cannot.

13.5 What a seller genuinely loses

Stated plainly, because a section claiming parity with hosted analytics platforms would be false:

- **No cross-site retargeting.** There is no shared identifier to follow one visitor between stores — the pseudonymous subject key of §13.3 is scoped to a single store by construction.
- **Campaign-level, not person-level, attribution.** A seller learns which channel a cohort arrived from, not which visitor did what before converting.
- **No individual session replay.** Nothing records one visitor's path through a store; there is no object type that could be queried for it.
- **Coarser fraud signals on non-reversible rails.** §13.6.

- **Noisier numbers for small sellers.** Aggregation needs volume to be both private and accurate; a store with ten visitors a day gets a bucketed count that swings hard on small changes. This cost is real, and it falls hardest on exactly the sellers this design exists to serve — a one-person store cannot buy its way to a bigger sample the way a hosted platform's shared infrastructure effectively lets its tenants do.

What a seller keeps: funnel counts, traffic sources at campaign granularity, geography sufficient for shipping and tax, product-level performance, and full order detail for customers who actually bought.

13.6 Fraud without raw IP

Most of what a raw IP address is used for in fraud scoring is a proxy for three separable questions:

Question	Answered by	Without identity because
Is this a bot?	anonymous rate-limiting credentials	a visitor proves they are within a rate budget without revealing who they are
Is this the same actor as last time?	the per-store pseudonymous key (§13.3)	linkage holds within one store and does not cross to any other, so it cannot become a cross-store tracking identifier by accretion
Is this location consistent with the payment method?	the payment provider	this is data the provider already holds to run its own rail; the protocol does not duplicate it

The honest residual: a merchant on a final, non-reversible rail with no provider-side fraud tooling has weaker fraud protection than one settling behind a card network's chargebacks and issuer risk scoring. That gap is real, nothing in this section closes it, and it is a reason to choose the rail class deliberately (§9.3) rather than by default.

13.7 This section is unevidenced

Three research passes across this document's drafting targeted privacy-preserving analytics and fraud specifically, and returned no verified findings on any of it. Everything above — the grant shape, the aggregate object, the claim that per-store pseudonymous keys resist cross-store linkage, the claim that small-seller noise is the dominant cost — is design reasoning: a shape argued from the substrate's own properties, not a measured result, not a deployed precedent at this scale, and not a citation of prior art shown to hold against an adversarial analyst. Read every fidelity claim in this section as a hypothesis this document has not tested, not a finding it has confirmed.

What §13.3 and §13.6 intend to profile, named as intentions rather than as evaluated fits:

- **Prio-style aggregation** — client-side secret-sharing so a collecting node learns only the aggregate, never an individual contribution — for `stage_counts` and `geography` in §13.4.
- **Privacy-preserving attribution work from the browser vendors** — the aggregate-and-noise pattern behind the various post-third-party-cookie attribution proposals — for campaign-level attribution without a cross-site identifier.
- **Anonymous credential schemes** for the rate-limiting credential of §13.6, so a visitor can prove eligibility without a name attached to the proof.
- **Oblivious HTTP (RFC 9458)** where IP-level unlinkability is wanted at the transport, ahead of whatever the application-layer grant already withholds.

None of these has been evaluated against this section's specific object shapes. Naming them states what this section intends to reuse rather than invent; it is not a claim that the fit has been checked.

13.8 Open

- Whether the per-store pseudonymous subject key (§13.3) needs a proof of non-linkability beyond "it is a different key per store" — a naive derivation (store key plus a buyer secret, hashed) could still be correlated by a seller who also runs, or buys data from, an index (§2.6).
- Whether `GRANT_LIFETIME`'s 24-hour default (§19.6) is the right value for analytics specifically, or was carried over from the same round-number instinct that set `FUND_TIMEOUT` to 24 hours for an unrelated reason.
- Whether aggregate telemetry needs formal noise on top of bucketing, or coarse buckets are sufficient protection at the traffic volumes real stores see — unresolved because those volumes are themselves unmeasured (§13.5).
- Whether a revoked grant obliges a seller to remove its already-folded contribution to a past aggregate, given that folding is one-way by construction and "stop disclosing" may not be the same operation as "undisclose".
- Whether the grant object needs a machine-readable purpose field (marketing, fraud, fulfilment-adjacent measurement) so a buyer's node can grant narrowly, or whether `fields` alone is granular enough.

14. Anti-abuse

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

14.1 Scope

Listing spam, fake stores, review manipulation, Sybil identities, and resource exhaustion.

14.2 The structural position

A seller can flood only **their own** feed, which only their own followers and holders pay for and may stop serving at will. There is no shared feed to spam and no fan-out amplification: a bad actor publishing garbage grows the cost of following that one seller, not the cost of running the network.

	Shared feed (a marketplace)	**A seller's own signed feed (TRACT, §2)**
Who pays to store and serve one seller's spam	every holder of the shared feed, whether or not they follow that seller	only holders who chose to hold that seller's feed
Blast radius of one bad actor	the whole marketplace's index and storage	that seller's own followers
Can a holder opt out of just the bad actor	no — the feed is shared	yes, at any time (§14.3)

This is inherited from the substrate's public-object model (§0.3, capability ②) and is why catalogue spam is a genuinely weaker threat here than on a platform with one shared listings feed. It is worth stating plainly rather than leaving it implied: this is a real structural advantage, not a rhetorical one, and it holds regardless of how the rest of this section's open questions resolve.

14.3 Holder admission policy

A holder (§0.4.1) decides what it stores and serves, and that decision is where most abuse is actually stopped, before an object ever reaches an index:

- **Object-size ceilings.** `MAX_PRODUCT_RECORD` (64 KiB), `MAX_OFFER` (16 KiB), `MAX_RATE_CARD` (1 MiB), `MAX_REVIEW_BODY` (8 KiB) — the full set is §19.5. A holder that enforces these bounds cannot be made to store an unbounded object by a publisher who simply declines to stop writing.
- **Per-publisher storage quota.** A ceiling on total bytes a holder will keep for one identity key, independent of how many individual objects that key publishes. Size limits bound one object; a quota bounds one publisher writing many small, individually-legal ones.
- **Feed-append rate.** A ceiling on how fast one feed may grow, so a compromised or malicious key cannot exhaust a holder's storage or bandwidth faster than the holder can react. This is a holder-side policy parameter, not yet collected in §19, and doing so is listed at §14.8.

None of these is a protocol requirement on what a seller may publish — a seller's own feed is theirs to fill (§0.4.1). They are requirements on what a *holder* is willing to carry, and a holder is free to set them tighter than any floor this document proposes.

14.4 Index admission policy, and the refusal rule

An index (§0.9 glossary; §2.6) decides independently what it aggregates. **Refusal to serve or index is a policy decision, never a protocol-level takedown.** No holder can compel another holder to serve an object, and no index can compel a holder to stop serving one, or compel another index to agree with its own admission choices. A disagreement between an index and a feed always resolves in favour of the feed (§0.4.1): an index that refuses to list a seller has removed itself from that seller's discovery, not removed the seller.

The corollary is that indexing power concentrates the same way holding power does not: refusing to index is cheap, individually reversible, and produces no shared record other than the index's own choice. §0.4.1 already discloses the consequence — a dominant index becomes a de facto content-policy gatekeeper regardless of what this document permits — and that disclosure is §2.6's to own; this section's obligation is narrower: the *mechanism* of refusal is policy, at every layer, by construction, never a network-level removal.

14.5 Cold-contact economics

A sealed order from a stranger (§7, §16.6) is, structurally, an unsolicited first-contact message — exactly the case the substrate's own anti-abuse tiers were built to price, not a new problem TRACT introduces. Two substrate properties already bound the cost of spamming orders at a seller:

- the **mailbox** a sleeping seller wakes through is short-TTL and content-blind (§0.3, capability ④) — nothing sent to it persists past that TTL, and nothing about its content is visible to whoever operates it, so it cannot be used as free bulk storage for junk orders the way an open inbox can;
- **wake** delivery is content-free and sender-blind (§0.3, capability ⑤) — a wake signal carries no payload and does not identify who sent it to the transport, so it gives an attacker no cheap way to fan a single spam campaign out across many sellers' wake paths at once.

A seller still decides, per §14.3's admission logic applied to its own inbound path, how much unsolicited contact from unrecognised keys it accepts before requiring some form of prior relationship (a follow, a previous order) — that policy is the seller's, not the protocol's, for the same reason holder admission is (§14.3).

14.6 Review manipulation: what purchase attestation does and does not prevent

§10.2 specifies purchase attestation: a proof, issued by the seller or an escrow operator, that a review's author actually transacted — verifiable by anyone, and it makes ballot-stuffing cost a real transaction rather than a keystroke. As an anti-abuse mechanism specifically, it is worth being exact about its edges, because the closest deployed relative of this design measured them.

OpenBazaar's reputation system was self-published, unweighted reviews with no banning authority, and **one vendor faked 60% of measured sales value** (§21.3, §21.6). Purchase attestation is strictly stronger than that — it binds a review to a signed transaction record — but two of OpenBazaar's failure conditions are not addressed by attestation and survive unchanged in this design as currently written:

1. **Self-dealing produces genuine attestations.** A seller transacting with a key it also controls generates a real, verifiable proof of purchase. Attestation raises the cost of faking a review and leaves a signed trail; it does not, and cannot, establish that the counterparty was an independent party.
2. **Escrow-issued attestations are scarcest exactly where they would matter most.** Escrow is opt-in (§9.5), and the measured OpenBazaar outcome was that bad actors simply declined the protection that would have constrained them — declining it was free, and the actors most worth constraining were the ones with the clearest incentive to decline. §9.5a and §10.3a already carry this as a measured cost of optionality rather than a hypothetical; this section inherits it as an anti-abuse limit rather than re-deriving it.

Both are stated in full in §10.3a and §10.4, which this section defers to rather than duplicates; what belongs here is the anti-abuse framing — attestation is admission control on *who may credibly claim to have transacted*, not a guarantee that the transaction was arm's-length.

14.7 Sybil and whitewashing

A new identity key has no history. Discounting new keys — weighting an unfamiliar seller, courier, distributor or reviewer down until it has accumulated some track record — is the only defence this design has against a bad actor simply generating a fresh key after being caught, and it is a blunt one: it penalises every legitimate newcomer exactly as much as it penalises a whitewasher, because the two are indistinguishable at the point a key first appears. There is no resolution to that which does not require a party empowered to vouch for or gate new identities — which is an authority, and reintroducing one to solve Sybil resistance would concede the point the rest of this document is built to avoid.

The achievable Sybil-cost floor on a signed-feed substrate is an open question, not a solved one this section is understating: §21.8 lists it as entirely unresearched by either literature pass behind this document. Whether buyer counter-signatures, transaction-cost binding, or purely local web-of-trust scoring actually raise that floor is unknown rather than assumed, and an index's weighting policy is where whatever judgement is currently applied actually lives — different indexes reaching different answers here is the design (§10.3), not a gap in it.

14.8 Honest limits

- Attestation makes a fake review or a fake sale expensive and leaves a signed trail. It does not make either impossible, and §14.6 states exactly where it stops.
- Holder and index admission policy (§14.3, §14.4) is enforced locally and voluntarily; nothing compels any two holders or indexes to agree on where the line sits, and a seller excluded everywhere still has a valid, undeleted feed.
- Feed-append rate and per-publisher storage quota are described here as concepts a holder needs; neither has a proposed default in §19 yet.

14.9 Open

- Whether feed-append rate and per-publisher storage quota belong in §19 as suggested defaults, or are inherently holder-local policy that a shared table would misrepresent as protocol-level.
- The achievable Sybil-cost floor on a signed-feed substrate (§21.8) — unresearched, and blocking a more specific answer than "discount new keys" for §14.7.

- Whether cold-contact policy (§14.5) needs a machine-readable declaration on a seller's feed — so a buyer's node knows before sending whether an unsolicited order will even be looked at — or is better left as an undeclared, seller-side heuristic.

15. Conformance

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

15.1 Scope

The four profiles, the auditable fail-closed set, how a profile is advertised, and the current state of the conformance vectors.

15.2 The four profiles

A node need not implement everything to be conformant. The profile it claims is a floor on what it must do — not a hint about what else it probably does — and what a profile deliberately excludes is as much a part of its definition as what it requires.

Profile	Builds on	Requires	Explicitly does not require
catalogue-only	—	publish and/or serve `ProductRecord`, `Offer`, `RateCard` and `CapacityRecord` objects (§2, §8.2) over the public quadrant; honour feed-head ordering, so an older signed head is never accepted over a newer one already observed (§2); serve derived index queries without asserting authority over a seller's own feed (§2.6)	accepting a sealed `Order` (§7); computing a delivery total (§8); holding or brokering funds (§9); rendering a storefront (§12)
transacting	catalogue-only	send and receive sealed `Order` objects and drive every transition of §18.3's state machine, including its timeouts; re-evaluate a cart line's live availability before checkout (§6.2); apply the substrate's cold-contact tiers to an unsolicited order (§14.3)	computing a route or a shipping total locally — a transacting node may present an order without one, or delegate the computation to a routing-profile peer
routing	transacting	compute a leg's price and transit estimate from a published `RateCard` object locally, without a live call to the carrier (§8.2); evaluate the small consolidation candidate set (§8.3)	being a carrier or a distributor itself — those are roles (§0.4.1), independent of which profile a node advertises
gateway	— (may stand alone, or combine with any of the above)	storefront rendering (§12) or settlement and escrow (§9.5), or both — the two bundle only because they share the same commercial and legal standing (§0.4.2), not because one implies the other; whichever it does, the matching fail-closed obligations of §15.3 apply	the half it does not do — a storefront-only gateway need not settle, and an escrow-only gateway needs no storefront, because two TRACT-native parties never need a gateway to transact at all (§0.4.2)

None of the four is "complete TRACT," and none is meant to be. This mirrors §0.4.1's roles: a seller needs a keypair and a box that is up, not a delivery-routing engine or a payment-provider relationship, and a profile boundary exists precisely so a catalogue-only node never has to build — or fake — the parts of the protocol it has no reason to run. Forcing every implementation to speak every profile would recreate the single do-everything platform the rest of this document exists to avoid.

The "builds on" column is a dependency, not a menu restriction. A node cannot claim routing without transacting — computing a route only matters once an order exists to attach it to — but a node can claim cata-

logue-only and gateway together (a storefront in front of its own catalogue, settling nothing) without ever claiming transacting.

15.3 The fail-closed set

Every security-relevant failure in this document is refused outright, or surfaced to both parties as an explicit choice; nothing degrades silently into an outcome that merely looks like success. Four of these are singled out because their violation is evidence of a defect in the implementation, not a business outcome an operator chose: escrow-scope intersection failure (§9.4), rail-class substitution without both parties agreeing (§9.3), origin isolation (§12.3), and the public-quadrant personal-data prohibition (§0.5.1).

The table indexes every failure of this kind that §17.4 has already assigned a code to. The owning clause is authoritative; this table is a pointer to it, not a restatement — a change to what a rule actually requires happens at the clause cited, never here.

Violation	Owning clause	Code (§17.4)
Personal data present in a public-quadrant object	§0.5.1	`0x0102`
Sealed and public object type confusion at decode	§16.4	`0x0101`
Offer missing a required axis	§2.4	`0x0201`
Two `money` values of different currencies combined without a disclosed conversion	§5.3	`0x0401`
A sale that would exceed a seller's combined quota across partitioned replicas	§6.2, §6.4	`0x0501`
A route total exceeding the representable range of a `money` minor-units integer	§8.2	`0x0701`
A rate card used to compute a quote past `RATE_CARD_MAX_AGE` without disclosing its age	§8.2, §19.6	`0x0702`
`EscrowScope` intersection between buyer and gateway is empty	§9.4	`0x0801`
Rail class substituted without a fresh party agreement recorded on the order	§9.3	`0x0802`
`Review` lacking a valid purchase attestation	§10.2	`0x0901`
Place of supply unresolved at the point tax treatment is computed	§11.2	`0x0A01`
Two stores served from one gateway origin	§12.3	`0x0B01`

15.4 Advertising and negotiating a profile

Profile advertisement rides the substrate's capability-token mechanism (§0.3, capability ④) rather than a bespoke TRACT-side negotiation. A node's advertised profile set is a signed, versioned announcement, and a stale, lower announcement is rejected on the same monotonic version basis the substrate already defines for every capability token — a node cannot be tricked into believing a peer has silently dropped back to a narrower profile than it last advertised.

The rule that follows is inherited unchanged from the substrate rather than restated in TRACT's own words: a capability absent from a peer's current announcement is a **fact about that peer**, never a fault to be worked around, retried past, or read as a temporary condition. A buyer's node that finds a seller advertising catalogue-only does not treat the missing transacting profile as an error to route around — there is nothing to route around. It is the honest current shape of that seller: browsable, not yet buyable through this protocol.

The consequence for conformance is symmetric with §15.2's "does not require" column: a conformance harness tests a node against the profiles it claims, never against the profiles it does not. `conformance/SUIT-E.md`'s TRACT-PROFILE-01 and TRACT-PROFILE-02 cases turn exactly this into assertions — a catalogue-only node must refuse an inbound sealed order rather than silently accept and ignore it, and a node advertising transacting without routing must not compute delivery routing on the buyer's behalf while implying that it does.

15.5 Vectors: the honest status

There are zero conformance vectors in this repository. `conformance/SUITE.md` lists 39 planned cases across the four profiles above, organised by the section each pins; every one is marked PLANNED, and none is backed by a byte-exact vector, because §16 (wire format) is not yet normative — there are no committed object shapes for a vector to check against. Freezing one early would invent bytes no numbered section actually specifies, or get silently invalidated the moment §16 lands with a different shape and nobody notices, because a vector nothing runs cannot fail a build (`conformance/README.md`).

`make coverage` reports this honestly rather than projecting a percentage:

```
conformance: 39 case(s) planned, 0 runnable (no vectors until §16 is normative — see
conformance/README.md)
```

That number moves only when a case is added to or removed from `conformance/SUITE.md`. It does not become "0 runnable" less true until a vector is derived from normative §16 text by the discipline `conformance/README.md` states — never exported from a reference implementation and back-filled here.

15.6 Open

- Whether a fifth, distinct profile for index-building behaviour is worth advertising, given §0.4.1 already treats building one as available to any node without registration. The four profiles above describe transactional capability, not derived-data capability, and TRACT-CAT-02 and TRACT-TRUST-02 already assume index behaviour that no profile currently names.
- Whether profile composition (transacting implying catalogue-only, routing implying transacting) needs to be enforced by the capability-token grammar itself, or stays a conformance-suite check layered on top of an announcement mechanism that does not itself understand dependencies.
- Whether a signed capability announcement is sufficient evidence of conformance, or whether the suite needs a live-probe component. An announcement authenticates to a key, not to the behaviour behind it, and TRACT-PROFILE-02 is written against the latter.
- What "conformant" means for a gateway that advertises both storefront and settlement but only actually delivers one. §15.2 permits claiming either half alone; the suite does not yet have a case for a gateway that claims both and only honours one.

16. Wire format

> **Status: normative, frozen at v0.** This section defines bytes, and unlike the sections around it, it is no longer a proposal. The key words MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are to be interpreted as in BCP 14 (RFC 2119, RFC 8174).
>
> **What freezing commits to.** A change to any shape below changes what an implementation puts on the wire, so it is a MAJOR version change under `GOVERNANCE.md` — not a correction. That is the point of freezing: §15's conformance vectors cannot exist against a moving target, and a second implementation cannot prove it matches this document rather than matching the reference one until there is something fixed to match.
>
> **One correction was made after freezing, before anything depended on it.** `Order` carried no jurisdictional anchors, while §11.3 stated that every order names them — a contradiction the first conformance vectors surfaced within hours of the freeze, which is precisely what a vector corpus is for. The fields were added rather than the gap recorded, because freezing something known to contradict another section preserves the appearance of stability and nothing else.

> **What it does not claim.** Every object here is implemented and exercised end to end in the reference implementation, which is evidence the shapes *compose* and is **not** evidence they are *right* — an implementation and a grammar derived from each other only prove they agree with one another (`conformance/README.md`). Freezing is a commitment to stability, not a claim of correctness, and §16.8 still lists what is undecided **above** the byte level.

16.1 Scope

The byte-level definition of every TRACT object: what is encoded, in what order, with which keys, and which of those keys a decoder may not tolerate.

16.2 Conventions inherited, not reinvented

Deterministic CBOR (RFC 8949 §4.2). Integer-keyed maps, keys assigned per object type from 1, keys ≥ 64 reserved for extension. A decoder MUST reject an unknown key in a **signed** object, failing closed; it MAY ignore unknown keys ≥ 64 in an **unsigned** object. Domain-separation tags on every signing preimage. Content addresses carry a multihash-style agility prefix.

TRACT introduces no new hash construction, no new signature framing, and no new address scheme. All four come from the DMTAP substrate. An implementation MUST NOT invent one; a construction appearing here would be a defect in this document.

16.3 Shared primitives

```
content-address = bytes         ; multihash-style prefix ‖ digest (substrate §18.1.5)
identity-key    = bytes         ; the public half of an IK; also a courier, distributor or
gateway
ts              = int           ; milliseconds since the Unix epoch
country         = tstr .size 2  ; ISO 3166-1 alpha-2
currency        = tstr .size 3  ; ISO 4217

money = {
  1 => int,        ; minor_units — NEVER a floating-point value (§16.7)
  2 => currency,
}
```

16.4 The structural rule — two type families, not one with a flag

Public and sealed objects are **separate type families**. A public object MUST NOT carry a name, address or contact detail (§0.5.1), and the grammar below is written so that it cannot: the prohibition is enforced by the productions, not by reviewer discipline, because published objects are irrevocable and an erasure right cannot be satisfied against them after the fact.

Two consequences the grammar has to carry, rather than leaving to convention:

- **A locality, never an address.** Public objects that reference a place (`PlaceRef`, `CapacityRecord`) carry a country and a coarse locality only. There is no street-address production in the public family at all, so "just add a line" is a grammar change a reviewer sees rather than a field a contributor adds.
- **No boolean discriminator.** A decoder is always in exactly one mode — expecting a public object or a sealed one — and the two are made non-confusable at the *content-address* level by distinct domain-separation tags, in the same way the substrate separates its own sealed and public manifests. There is no

`sealed: true` field an attacker could omit, because a flag can be dropped and a domain-separation tag cannot. A decoder expecting one family and handed the other MUST reject it rather than coerce.

16.5 Public objects

16.5.1 `ProductRecord` — what a thing is

Belongs to nobody. Two publishers whose canonicalised records encode identically converge on one content address (§2.2), with the caveat §2.2a records about how weak that is in the real case.

```
ProductRecord = {
  1 => tstr,                    ; name        canonicalised (§2.3)
  2 => tstr,                    ; description canonicalised; may be empty
  3 => [* Attribute],           ; attributes  sorted, deduplicated, keys casefolded
  4 => [* IdentityRung],        ; identity    weakest first
  ? 5 => content-address,       ; group       the ProductGroup this varies within
  ? 6 => [* content-address],   ; components  bundle members; may reference other sellers'
records
}

Attribute = { 1 => tstr, 2 => tstr }    ; key (casefolded), value

IdentityRung = ContentAddressRung / ClaimedExternalRung / ManufacturerSignedRung
ContentAddressRung     = { 0 => content-address }
ClaimedExternalRung    = { 1 => tstr, 2 => tstr }   ; scheme ("gtin", "mpn"), value —
UNVERIFIED
ManufacturerSignedRung = { 2 => identity-key }      ; the brand's own key
```

The middle rung is a **claim and nothing more**: anyone can assert any GTIN, so an index MUST treat it as an advisory join key and MUST NOT treat it as authority (§2.3). Squatting is expected rather than prevented.

16.5.2 `Offer` — one seller's claim to supply

```
Offer = {
  1 => Item,
  2 => Availability,
  3 => Fulfilment,
  4 => Consideration,
  5 => [* country],            ; sell_to — territories this offer may be accepted from
  6 => ts,                     ; published
}

Item = { 0 => content-address }                              ; product
     / { 1 => content-address, 2 => content-address }        ; variant-of-group: group, vari-
ant
     / { 3 => content-address }                              ; service
     / { 4 => content-address }                              ; right / licence
     / { 5 => content-address }                              ; capacity

Availability = { 0 => StockSignal }
             / { 1 => tstr, 2 => uint }        ; time-slots: RFC 5545 payload, slot minutes
             / { 3 => uint, 4 => tstr }        ; capacity-per-interval: capacity, RFC 5545
recurrence
             / { 5 => null }                   ; unlimited
             / { 6 => uint }                   ; made-to-order: lead days

StockSignal = { 0 => uint } / { 1 => null } / { 2 => null } / { 3 => null }
            ; exact(n) / in-stock / low / out-of-stock — a band is publishable instead of a
number,
            ; because exact stock is commercially sensitive and browsing does not require it

Fulfilment = { 0 => [* country] }             ; ship: destinations served
           / { 1 => PlaceRef }                ; collect
           / { 2 => null }                    ; digital-grant
           / { 3 => PlaceRef }                ; perform-at-place — the venue
           / { 4 => null }                    ; perform-remote
           / { 5 => PlaceRef / null }         ; access-grant, physical or not
           / { 6 => PlaceRef, 7 => uint }     ; return-required: place, term days

PlaceRef = { 1 => country, 2 => tstr }        ; country, LOCALITY — never a street address
(§16.4)

Consideration = { 0 => money }                          ; fixed
              / { 1 => [+ PriceTier] }                  ; tiered / volume
              / { 2 => money, 3 => tstr }               ; recurring: amount, RFC 5545 RRULE
              / { 4 => tstr, 5 => money }               ; metered: dimension, unit price
              / { 6 => money, 7 => money }              ; deposit + balance
              / { 8 => null }                           ; quote-required (RFQ; also B2B con-
tract pricing)

PriceTier = { 1 => uint, 2 => money }         ; min_qty, unit_price
```

The load-bearing detail. `Fulfilment` is not merely logistics: it is the only object that knows where a supply happens, and §11.2 derives the tax anchor from it. Ship → destination; collect / perform-at-place / return-required → the stated place; perform-remote and digital-grant → buyer residence; access-grant → whichever, depending on whether it names a place. An implementation MUST derive it from this object and MUST NOT accept it as a separate argument that could disagree; one resolving place of supply from the parties' countries instead will be plausibly and consistently wrong about every event held abroad.

16.5.3 `RateCard` and `CapacityRecord` — published, so routing is computed not quoted

```
RateCard = {
  1 => identity-key,            ; carrier — a multinational and a neighbour with a van, iden-
tically typed
  2 => [* country],             ; serves
  3 => [* Zone],
  4 => uint,                    ; dim_divisor — billable = max(actual, L*W*H / divisor)
  5 => uint,                    ; surcharge_pct
  6 => [* tstr],                ; excluded_categories
  7 => ts,                      ; published — cards drift; a stale one is not replayed
silently
}

Zone          = { 1 => uint, 2 => [+ WeightBracket], 3 => uint }  ; id, brackets, transit_-
days
WeightBracket = { 1 => uint, 2 => money }                          ; max_grams, price

CapacityRecord = {                                ; a distributor's published consolidation
offer
  1 => identity-key,            ; distributor
  2 => country,
  3 => tstr,                    ; locality — coarse (§16.4)
  4 => money,                   ; storage_per_day
  5 => money,                   ; handling_fee
  6 => uint,                    ; slots
  7 => [* tstr],                ; excluded_categories
}
```

A rate card is a **claim by its publisher**, and a transit figure inside it is an estimate. §8.4 records that some carriers restrict redistribution of negotiated rates, which is why an offer may instead declare that a live quote is required.

16.5.4 `EscrowScope` — what an operator can lawfully serve

```
EscrowScope = {
  1 => identity-key,            ; operator
  2 => [* country],             ; buyer_countries
  3 => [* country],             ; seller_countries
  4 => [* country],             ; supply_countries — checked against place of supply, not the
parties
  5 => [* currency],
  6 => [* RailClass],
  7 => money,                   ; max_order_value — usually a KYC threshold
  8 => [* tstr],                ; excluded_categories
  9 => [* tstr],                ; authorities claimed — prose, because regulators share no
schema
}

RailClass = 0 / 1              ; 0 = custodial-reversible, 1 = non-custodial-final
```

`supply_countries` is a separate field from the party countries deliberately: two trades with identical buyers, sellers, currency and rail can differ only in where the supply happens, and that difference alone can put one of them outside an operator's licence (§9.4).

16.5.5 `Review` and `PurchaseAttestation` — the bounded exception

```
Review = {
  1 => Subject,
  2 => identity-key,            ; author — a PER-SUBJECT pseudonymous subkey, never the root
IK
  3 => uint,                    ; score, 0..5
  4 => tstr,                    ; body
  ? 5 => PurchaseAttestation,
  6 => ts,
}

Subject = { 0 => content-address }   ; product
        / { 1 => identity-key }      ; seller
        / { 2 => identity-key }      ; distributor
        / { 3 => identity-key }      ; courier

PurchaseAttestation = {
  1 => 0 / 1,                   ; attestor: 0 = seller, 1 = escrow operator
  2 => identity-key,            ; issuer
  3 => content-address,         ; order — the SEALED order's address only; never its contents
  4 => ts,
}
```

A review is the one public object signed by a natural person, and §10.4 bounds it rather than exempting it. Two things the grammar carries: the author is a per-subject subkey so reviews are not trivially linkable across sellers, and the attestation references a sealed order **by address only** — nothing about what was bought crosses into the public quadrant.

`body` remains free text a person could type an address into. The grammar cannot prevent that; §10.4 and the client requirements have to, and pretending otherwise would be exactly the kind of overclaim §16.4 exists to avoid.

16.6 Sealed objects

Never published. Held at the two endpoints, where deletion is meaningful.

```
Order = {
  1 => identity-key,            ; buyer
  2 => identity-key,            ; seller
  3 => [+ OrderLine],           ; THIS seller's lines only — never the whole cart
  4 => money,                   ; total
  5 => OrderState,
  6 => ts,                      ; placed
  7 => Anchors,                 ; the four jurisdictional anchors (§11.2)
  8 => Responsible,             ; who is answerable, and for what (§11.3)
  ; buyer name, delivery address and contact details are carried here, in the sealed family,
  ; and have no production in the public family at all (§16.4)
}

; §11.2's four anchors, carried on the order because they are the facts a regulator asks for
and
; because three of them cannot be recomputed later: buyer residence is disclosed at order
time and
; nowhere else, and place of supply depends on the fulfilment variant as it stood when the
order
; was placed, not as the offer reads today.
Anchors = {
  1 => country,                 ; seller_establishment
  2 => country,                 ; buyer_residence
  3 => country,                 ; place_of_supply — derived from Fulfilment (§4), never sup-
plied beside it
  ? 4 => country,               ; delivery_destination — absent when nothing moves
}

; §11.3. `facilitator` present iff a gateway settled the payment: that presence is the
; marketplace-facilitator hook, and its absence is equally load-bearing, because a self-host-
ed
; seller taking direct payment is never a facilitator (§21.11.2).
Responsible = {
  1 => identity-key,            ; seller_of_record
  ? 2 => identity-key,          ; facilitator — the gateway, if it settled
  ? 3 => identity-key,          ; importer_of_record
  ? 4 => Representative,        ; in-region responsible person, where a regime requires one
}

Representative = { 1 => country, 2 => identity-key }   ; region covered, who

OrderLine  = { 1 => content-address, 2 => identity-key, 3 => uint }   ; offer, seller, quan-
tity
OrderState = 0 / 1 / 2 / 3 / 4 / 5 / 6 / 7 / 8
           ; draft / placed / accepted / declined / countered / fulfilling / delivered /
           ; closed / cancelled  (§18)

PaymentAttestation = {
  1 => identity-key,            ; payer
  2 => identity-key,            ; payee
  3 => content-address,         ; order
  4 => money,
  5 => RailClass,               ; determines the buyer's recourse — never flattened to a
boolean
  6 => tstr,                    ; external settlement reference, opaque
  7 => ts,
}
```

One order per seller is a grammar-level property, not a client convention. `Order` names a single seller and MUST carry only that seller's lines, so a cross-seller order is not expressible. The whole-cart view exists on the buyer's device and nowhere else (§6.1).

`PaymentAttestation` MUST carry a *reference* only — never funds, and never card data. The protocol conveys that a payment happened; it does not move money (§9.2).

16.7 Encoding rules that exist because of a specific failure

- **Money MUST be minor units and a currency code, never a float.** A rounding error in a signed object cannot be corrected after the fact — the signature covers the wrong number.
- **Arithmetic across currencies MUST be refused, never coerced.** A silently converted total is a wrong total that looks right, and it gets carried into an order (§5.3).
- **A decoder that finds an unrecognised field in a signed object MUST reject it**, rather than ignoring it. The alternative lets a signer and a verifier disagree about what was signed.
- **`ts` MUST be milliseconds since the Unix epoch**, subject to the substrate's clock-skew tolerance. A timestamp is for display and ordering; where order must be authoritative it comes from a feed's sequence, never from a clock.

16.8 Open

- **Whether `Offer` carries its own signature or inherits authenticity from its position in a signed feed.** The substrate's feed head transitively commits to every entry, which makes a per-object signature redundant — and makes an offer quoted out of its feed unverifiable. The trade is real and undecided.
- **Whether `sell_to` being empty means "unrestricted" or "malformed."** Unrestricted is convenient and is almost never what a seller means, given §11's in-region representative requirements. Leaning malformed.
- **Whether `Anchors.place_of_supply` should be recomputable or is authoritative as recorded.** It is stored because the fulfilment variant it derives from may change in a later offer revision, and a tax position should reflect the terms at order time. That makes it a fact a party asserts rather than one a verifier can check, which is a real weakening and the reason it is listed here.
- **Extension-key policy for the axis unions.** Each axis is a small closed set today. What a decoder does with an axis variant it has never seen — refuse, or preserve and refuse only on *acting* — is §17.4's tolerant-to-store / strict-to-act rule, and it needs stating here in grammar terms rather than only in prose.

17. Errors & registries

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

17.1 Scope

The `ERR_TRACT_*` code block, the responder-action vocabulary, an initial error table, and the extensible registries — everything a conformant implementation needs to name a failure the same way another implementation would, without either one inventing its own vocabulary for it.

17.2 Code format and the subsystem table

Every code is a 16-bit value `0xSSNN`: `SS` is the subsystem byte, `NN` is the code point within it — the layout the sibling DMTAP specification uses for its own registry (`21-errors-iana.md` §21.1), reused here for form only. TRACT allocates its own subsystem bytes rather than sharing DMTAP's table; a TRACT code and a

DMTAP code that happen to carry the same `SSNN` value name unrelated conditions in two different registries, never the same one.

This appendix introduces no new validation logic. Every code below traces to a rule some numbered section already states in prose — §0.5.1, §2.4, §5.3, §6.2, §6.4, §8.2, §9.3, §9.4, §10.2, §11.2, §12.3, §16.4 and §16.7 between them. Naming the rule and giving it a code does not make the rule normative before its own section is; the whole table is as provisional as those sections are, and expect renumbering as they firm up — not because the layout is arbitrary, but because the rule underneath a code might still change shape before the section stating it does.

SS	Subsystem
`0x00`	Reserved
`0x01`	Wire format — public/sealed structure (§16)
`0x02`	Catalogue (§2)
`0x03`	Availability & fulfilment (§3, §4)
`0x04`	Consideration (§5)
`0x05`	Cart (§6)
`0x06`	Order (§7)
`0x07`	Delivery (§8)
`0x08`	Settlement (§9)
`0x09`	Trust (§10)
`0x0A`	Jurisdiction (§11)
`0x0B`	Gateway (§12)
`0x0C`	Anti-abuse & feed continuity (§14)
`0x0D` – `0xEF`	Unassigned — reserved for future subsystems
`0xF0` – `0xFE`	Private use (experimental/vendor subsystems)
`0xFF`	Reserved

The grouping is not one byte per section number: §3 and §4 share a subsystem because the four axes (§2.4) are inspected together at the same points (an offer is rejected or accepted as a whole, never one axis at a time), and §13 and §15 have no subsystem byte yet because neither has a checkable object shape to raise a code against.

17.3 Responder action vocabulary

Every code below resolves to exactly one of a closed set of five actions. The set is closed deliberately — a sixth action invented ad hoc the day someone needs one is exactly how a registry like this drifts into having as many behaviours as it has codes.

Action	Meaning	When it applies
fail-closed-block	Refuse to proceed, and refuse to silently degrade into a lesser but still-plausible outcome. The object, order or trade stops here until the condition is fixed by the party who caused it.	The default for structural violations — a public object carrying personal data, a type-confused decode, an offer missing an axis — where continuing at all, in any reduced form, is the mistake.

Action	Meaning	When it applies
drop-silent	Discard without notifying the sender; to the sender this is indistinguishable from packet loss.	Reserved for conditions where notifying would itself leak information the sender should not have. TRACT's own table has no entry using it yet — the substrate already disposes of the case that would need it (an unrecognised message reaching a node under the substrate's own cold-contact handling, §14.3), and TRACT does not re-litigate that disposition.
rotate-retry	An internal recovery action — try a different path, a different replica, a refreshed copy of an object — with no user-visible failure unless the retries are exhausted.	Transient conditions, not structural ones. A rate-card fetch failing over a flaky connection is a substrate-level reachability condition (inherited, not a TRACT code); TRACT's own conditions are checks against content already received, which is why none of §17.4's rows use it today.
deny-policy	Reject per a deliberate, non-adversarial policy decision: the object is well-formed and could succeed elsewhere, but this responder's own declared scope or standard does not cover it.	An `EscrowScope` intersection that comes up empty, or a review an index declines to serve for want of attestation — both are facts about this responder's stated terms, not defects in what was presented.
halt-alert	Halt the affected process and raise a visible alert.	Reserved for conditions that indicate tampering or equivocation, not ordinary refusal. TRACT's own table has none yet: every condition §17.4 currently names is a bounded, explainable refusal, not evidence of an attack. The substrate's own halt-alert conditions (a broken identity chain, a split-view key-transparency log) already apply beneath TRACT unchanged and are not reproduced here.

17.4 Initial error table

Code	Name	Operation	Meaning	Retryable	Action
`0x0101`	`ERR_TRACT_TYPE_CONFUSION`	decode of any object (§16.4)	a sealed object is presented under a public-object schema, or a public object under a sealed one	No	fail-closed-block
`0x0102`	`ERR_TRACT_PERSONAL_DATA_PUBLIC`	decode or publish of `ProductRecord`, `Offer`, `RateCard`, `CapacityRecord`, or a storefront render bundle (§0.5.1, §16.4)	a public-quadrant object carries a field that identifies, or is linkable to, a natural person	No	fail-closed-block
`0x0201`	`ERR_TRACT_OFFER_AXIS_MISSING`	`Offer` decode (§2.4, §16.5.2)	one or more of the four axes — Item, Availability, Fulfilment, Consideration — is absent	No	fail-closed-block
`0x0401`	`ERR_TRACT_CURRENCY_MISMATCH`	consideration or route-total arithmetic (§5.3, §16.7)	two `money` values of different currencies are combined without an explicit, disclosed conversion	Conditional — a disclosed conversion supplies a single currency	fail-closed-block

Code	Name	Operation	Meaning	Retryable	Action
`0x0501`	`ERR_TRACT_OVERSELL_PREVENTED`	cart checkout against partitioned inventory (§6.2, §6.4)	a sale would exceed the combined quota remaining across a seller's replicas	No	fail-closed-block
`0x0701`	`ERR_TRACT_ROUTE_TOTAL_OVERFLOW`	route-total computation (§8.2, §16.7)	summing leg prices would exceed the representable range of a `money` minor-units integer	No	fail-closed-block
`0x0702`	`ERR_TRACT_RATE_CARD_STALE`	rate-card use for local quoting (§8.2, §19.6)	a `RateCard` older than `RATE_CARD_MAX_AGE` is used to compute a quote without disclosing its age	Yes — refetch a current card	fail-closed-block
`0x0801`	`ERR_TRACT_ESCROW_SCOPE_EMPTY`	checkout scope intersection (§9.4)	the buyer's declared `EscrowScope` and the gateway's offered scope intersect on no valid combination of country, currency, rail class, value ceiling or category	No	deny-policy
`0x0802`	`ERR_TRACT_RAIL_CLASS_SUBSTITUTED`	rail selection at checkout (§9.3)	a `CustodialReversible` rail is replaced with `NonCustodialFinal`, or the reverse, without a fresh agreement recorded on the order	No	fail-closed-block
`0x0901`	`ERR_TRACT_REVIEW_UNATTESTED`	`Review` intake by an index (§10.2, §16.5.5)	the review carries no valid `PurchaseAttestation` from the seller or an escrow operator	No	deny-policy
`0x0A01`	`ERR_TRACT_PLACE_OF_SUPPLY_UNRESOLVED`	tax-anchor derivation (§11.2, §16.5.2 `Fulfilment`)	the Fulfilment variant does not yet resolve to a single place of supply — most often a multi-variant offer whose buyer choice is not yet recorded on the order	Conditional — resolves once the buyer's choice is recorded	fail-closed-block
`0x0B01`	`ERR_TRACT_ORIGIN_NOT_ISOLATED`	render-bundle serving (§12.3)	two stores would be served from the same normalised origin	No	fail-closed-block

Retryable means the same condition, presented again with nothing else changed, can succeed — "Conditional" marks the cases where success depends on an action outside the responder's own control (the buyer recording a choice, a fresher object being fetched), not on retrying the identical request. `0x0401`'s and `0x0701`'s reasoning is the same reasoning, run in two directions: a currency silently coerced, or a total silently wrapped, is a wrong number that looks exactly like a right one, and by the time it is carried into a signed order there is no downstream step that can tell the difference. Refusing both at the point of computation is cheaper than auditing every order for one afterward.

17.5 Extensible registries

Registry	What it enumerates	Where defined	Extension policy
Item kinds	product / variant-of-group / service / right-or-licence / capacity	§2, §16.5.2 `Item`	tolerant to store, strict to act (§17.6)
Availability variants	stock-signal / time-slots / capacity-per-interval / unlimited / made-to-order	§3, §16.5.2 `Availability`	tolerant to store, strict to act
Fulfilment variants	ship / collect / digital-grant / perform-at-place / perform-remote / access-grant / return-required	§4, §16.5.2 `Fulfilment`	tolerant to store, strict to act
Consideration variants	fixed / tiered / recurring / metered / deposit-and-balance / quote-required	§5, §16.5.2 `Consideration`	tolerant to store, strict to act
Rail classes	custodial-reversible / non-custodial-final	§9.3, §16.5.4 `RailClass`	a closed set of two, not an ordinary extensible registry — a third class is a specification change, because §9.3 makes the class part of what determines a buyer's recourse, and a registry addition here would be a substitution by another name
Tax treatment categories	— not yet enumerated	§11 (§11.3 commits to the field; its values are undecided)	tolerant to store, strict to act, once populated
Excluded-category vocabulary	free-text category strings a rate card, capacity record or escrow scope may exclude	§8, §9, §16.5.3, §16.5.4	advisory strings today, not a controlled list — see §17.7
External-identifier schemes	`gtin`, `mpn`, and others a publisher may claim	§2.3, §16.5.1 `ClaimedExternalRung`	advisory join keys only, never authority (§2.3); a scheme absent from this registry is preserved and surfaced as unknown, never treated as a match

17.6 Tolerant to store, strict to act

Every registry above except rail classes is extensible, and an unrecognised value must not be fatal to a generic index: the object is preserved, and the unrecognised field is surfaced as unknown rather than causing the object to be dropped. An index that refused to hold an offer merely because it used a Fulfilment variant from next year's registry update would make every extension to this specification a backward-compatibility break for every index already deployed.

A client that *renders or transacts* against a variant it does not implement is a different actor making a different decision, and it refuses rather than guesses. The two behaviours are independently required because a case that only checked one could pass an implementation that guesses exactly where it should refuse.

Guessing is worse than refusing here for the same reason a silently coerced currency or a silently wrapped total is worse than a refusal (§17.4): a guess produces an answer that looks exactly as valid as a correct one, and nothing downstream carries a marker saying it was a guess. A client that renders an unrecognised Fulfilment variant as if it were `collect`, because `collect` is the closest shape it understands, has just told a buyer a wrong place of supply with the same confidence it would state a right one (§11.2) — and the buy-

er has no way to tell the difference until a tax authority does. A client that refuses instead produces a visible boundary: the buyer learns their client does not yet understand this offer, which is a true and actionable fact, rather than a false and confident one.

17.7 Open

- Whether the registries of §17.5 live as tables in this document, or as a separate machine-readable file this document points to. A table reviewed alongside the prose that defines each variant is easier to keep honest; a separate file is easier for an implementation to consume without parsing markdown.
- The allocation policy for the `0x0D`–`0xEF` unassigned range and the `0xF0`–`0xFE` private-use range — Specification Required, first-come-first-served, or something else — is undecided. Mirroring DMTAP's policy is one option, not a commitment.
- Whether the excluded-category vocabulary should graduate from free text to a controlled list. Free text is expressive and needs no maintainer; a controlled list is machine-actionable across implementations but needs one, which is exactly the centralising pressure §2.6's index rule exists to keep out of this specification.
- Whether tax treatment categories can be enumerated at all before jurisdiction is researched. §11.3 commits to the field; populating it with real category values may have to wait on work this document does not yet consider done.

18. State machines

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

18.1 Scope

The state machines, collected in one place so they can be checked against §7's prose rather than inferred from it — and so that the question this section exists to force gets asked of every edge: **what happens when the counterparty never answers?**

That question is the whole point. A protocol between sovereign parties has no supervisor to notice a stalled trade, so a transition without a defined expiry is not an edge case, it is where money and goods sit indefinitely with nobody empowered to move them. Every table below therefore carries a timeout column, and "none" appears only where waiting forever is genuinely correct.

18.2 Offer

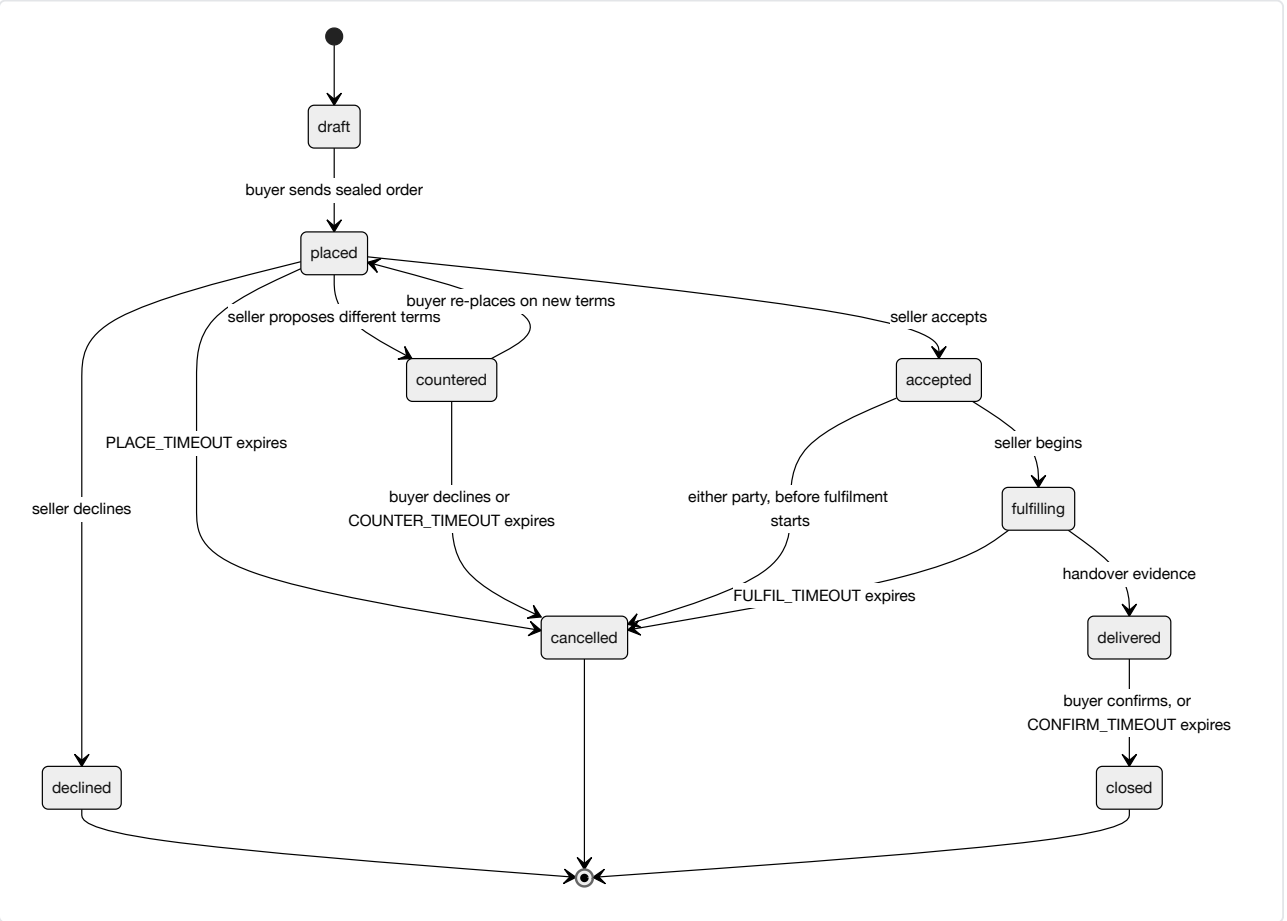
The simplest machine, and the only one with no counterparty: an offer is one seller's unilateral publication.

From	To	Trigger	Signed by	Timeout
—	`published`	the seller publishes	seller	none
`published`	`superseded`	a later offer replaces it	seller	none
`published`	`withdrawn`	the seller stops offering	seller	none

Withdrawal is not deletion. A published offer is content-addressed and irrevocable (§0.5), so `withdrawn` is a successor object stating the offer no longer stands, not an erasure of the original. A buyer holding a cached

offer learns it is withdrawn by fetching the seller's feed; until they do, they may present terms the seller has stopped honouring, which is why acceptance is the seller's (§18.3) and not automatic.

18.3 Order



From	To	Trigger	Signed by	Timeout → behaviour
`draft`	`placed`	buyer sends	buyer	none — a draft is local
`placed`	`accepted` / `declined` / `countered`	seller responds	seller	`PLACE_TIMEOUT` → `cancelled`. **A silent seller cancels; it never accepts.**
`countered`	`placed` / `cancelled`	buyer responds	buyer	`COUNTER_TIMEOUT` → `cancelled`
`accepted`	`fulfilling`	seller begins	seller	none
`fulfilling`	`delivered`	handover evidence (§18.4)	carrier or seller	`FULFIL_TIMEOUT` → `cancelled`, and escrow refunds (§18.5)
`delivered`	`closed`	buyer confirms	buyer	`CONFIRM_TIMEOUT` → `closed`. **A silent buyer closes; escrow releases.**

The two asymmetric defaults are deliberate and pull in opposite directions, which is the honest part:

- **Silence before acceptance cancels.** A seller who never answers must not be bound, and a buyer must not have funds committed against an order nobody acknowledged.
- **Silence after delivery closes.** A buyer who received goods and then stops responding must not be able to strand the seller's money forever by doing nothing. This is the one place the protocol favours the seller, and it does so because the alternative — funds held until a buyer affirmatively acts — makes non-response a free option to withhold payment.

`CONFIRM_TIMEOUT` is therefore the most consequential parameter in §19, and it is a genuine trade-off rather than a tuning detail: too short and a buyer with a real complaint loses recourse by being slow; too long and every seller carries the float. It is stated here so the choice is made visibly rather than inherited from whatever a first implementation happened to pick.

Cancellation is not symmetric with acceptance. After `fulfilling` begins, a unilateral buyer cancellation is a *request*, not a transition — goods may already be in a courier's hands, and §8 custody has its own machine (§18.4). What the buyer can always do is refuse to confirm, which is what routes the trade to a dispute rather than to `closed`.

18.4 Consignment

Physical custody, which is the one machine whose transitions correspond to something happening in the world rather than a message being sent.

From	To	Trigger	Signed by	Timeout → behaviour
—	`created`	seller books a leg	seller	none
`created`	`accepted`	carrier or distributor takes custody	**the receiving party**	`PICKUP_TIMEOUT` → `created` again, so the seller re-books
`accepted`	`in_custody`	held at a hub	custodian	`HOLD_TIMEOUT` → alert; consolidation is waiting on a slower seller (§8.3)
`in_custody`	`handed_off`	passed to the next leg	**both** outgoing and incoming	none
`handed_off`	`delivered`	final handover	recipient, or carrier proof-of-delivery	`TRANSIT_TIMEOUT` → `lost`
any	`lost`	declared	current custodian, or by timeout	terminal

Custody transitions are signed by the party taking custody, not the party giving it up. A handoff attested only by the sender proves someone *tried* to hand something over; attested by the receiver, it proves the chain actually moved. This matters because the whole value of the chain is being able to say who held the goods when they went missing.

And it proves transfer, not recoverability. §8.4 says this and §18 has to repeat it here rather than let a tidy state machine imply otherwise: `lost` is a terminal state with a signed history and no remedy attached. Where the goods went is answerable; getting them back, or being paid for them, is §9's problem or nobody's.

18.5 Escrow

Only present when both parties chose an operator whose scope covers the trade (§9.4).

From	To	Trigger	Signed by	Timeout → behaviour
—	`funded`	buyer pays	operator attests	`FUND_TIMEOUT` → order `cancelled`
`funded`	`held`	seller dispatches	operator attests	—
`held`	`released`	buyer confirms, or order reaches `closed`	operator	`CONFIRM_TIMEOUT` (§18.3) → `released`
`held`	`refunded`	order reaches `cancelled`, or ruling	operator	—
`held`	`split`	ruling	operator	—

From	To	Trigger	Signed by	Timeout → behaviour
`held`	`held`	dispute raised	either party	`DISPUTE_TIMEOUT` → **see below**

The edge with no good answer. A dispute where neither party will move is exactly what escrow exists for, and it is also where non-custodial programmatic escrow deadlocks (§9.6). A custodial operator resolves it by ruling, which is why it is an operator class at all — and that ruling is published as a signed object (§9.5), so an operator that rules badly accumulates a record of it.

For a **non-custodial** rail there is no such move available. The honest options are a timeout that defaults to one party — which is a policy choice favouring whoever it defaults to, not a neutral mechanism — or indefinite lock. This section does not pretend a third option exists. What it requires is that the choice is **disclosed before the trade**, because a buyer who learns at dispute time that no one can release the funds was mis-sold the arrangement.

18.6 What every transition carries

- **Which party signs it.** A transition nobody signed is not evidence of anything.
- **What evidence it produces**, and whether that evidence is public (an escrow ruling, §9.5) or sealed (an order transition, §16.6).
- **Its timeout, and the state that timeout leads to.** Never "expires" alone: expiring *into* somewhere is the whole requirement.
- **Whether it is reversible**, and by whom. Most are not, which is why acceptance and confirmation are the two places a party gets to think.

18.7 Open

- **Whether `CONFIRM_TIMEOUT` is a protocol parameter or an offer-level term.** A seller of perishables and a seller of furniture want very different values, which argues for the offer; a buyer comparing offers should not have to read a timeout to know their recourse, which argues for the protocol. Currently leaning protocol floor with an offer-level extension upward only.
- **Whether a dispute pauses `CONFIRM_TIMEOUT` or runs alongside it.** Pausing lets a bad-faith buyer stall indefinitely by disputing; not pausing lets a slow operator time out a real dispute into an automatic release.
- **Whether `lost` needs a distinct `disputed-lost`**, given that custodian and recipient may disagree about whether delivery happened at all.

19. Parameters

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

19.1 Why this is its own section

Parameters scattered through prose drift out of agreement with each other, and nobody notices until two sections cite different values for the same thing. Collecting them lets the linter check that a figure quoted in §N matches the table, and lets an implementer see the whole configuration surface at once instead of discovering it a section at a time.

The values below are **proposed defaults, not measurements**. Where a value is a genuine trade-off rather than an obvious floor, the trade-off is stated beside it — a number with no reasoning attached is a number the next person changes arbitrarily.

19.2 Order timeouts (§18.3)

Parameter	Proposed	Expires into	The trade-off
`PLACE_TIMEOUT`	72 h	`cancelled`	Long enough for a one-person seller to be away for a weekend; short enough that a buyer is not left guessing. A silent seller is never treated as having accepted.
`COUNTER_TIMEOUT`	72 h	`cancelled`	Symmetric with the above, since a counter puts the buyer in the seller's former position.
`FULFIL_TIMEOUT`	offer-declared	`cancelled`, escrow refunds	Cannot be one number: made-to-order declares lead days (§3), a download is instant. The availability axis already carries the expected duration, so this parameter is a *floor* on how long a buyer must wait before they may cancel — not the duration itself.
`CONFIRM_TIMEOUT`	**14 days**	**`closed`, escrow releases**	**The most consequential value in this table.** Too short and a slow buyer with a real complaint loses recourse by being slow. Too long and every seller carries the float on every order. 14 days is proposed because it sits near the cooling-off windows several consumer regimes use, which makes it defensible rather than arbitrary — but §11's research gap means that alignment is *asserted, not verified*.

19.3 Consignment timeouts (§18.4)

Parameter	Proposed	Expires into	Note
`PICKUP_TIMEOUT`	48 h	`created`	Returns to bookable rather than failing: a courier that never collected is a re-book, not a lost order.
`HOLD_TIMEOUT`	offer-declared	alert only	Consolidation waits on the slowest seller (§8.3). Raises an alert; does not cancel, because the buyer chose to wait.
`TRANSIT_TIMEOUT`	carrier estimate × 3	`lost`	Derived from the rate card's own published transit figure rather than a fixed number, since a same-city courier and a cross-border leg have nothing in common. The multiplier is the parameter; the estimate is the carrier's claim.

19.4 Escrow timeouts (§18.5)

Parameter	Proposed	Expires into	Note
`FUND_TIMEOUT`	24 h	order `cancelled`	An unfunded escrow holds nothing, so expiry costs nobody anything.
`DISPUTE_TIMEOUT`	operator-declared	operator ruling	**Custodial rails only.** A non-custodial rail has no ruling available, and §18.5 requires the resulting behaviour — default-to-one-party, or indefinite lock — to be disclosed before the trade rather than discovered at dispute time.

19.5 Object and serving limits

Parameter	Proposed	Rationale
`MAX_PRODUCT_RECORD`	64 KiB	A record describes a thing; images are separate content-addressed blobs.
`MAX_OFFER`	16 KiB	An offer is four axes and a few territories.
`MAX_RATE_CARD`	1 MiB	Zone tables are genuinely large. The one object where a big number is expected.
`MAX_REVIEW_BODY`	8 KiB	Also a **privacy** bound, not only a resource one: the smaller this is, the less room there is to type personal data into a public irrevocable object (§10.4).
`MIN_DIM_DIVISOR`	1000	Real carriers publish 5000 or 6000. A divisor between 1 and 1000 is not one any carrier uses, and it inflates the billable weight of every parcel computed from that card (§8.2). Zero is permitted and means “this carrier does not apply volumetric weight”.
`MAX_CART_LINES`	500	A cart is buyer-held (§6.1), so this bounds what a *seller* must be prepared to receive, not what a buyer may collect.

19.6 Rate and freshness

Parameter	Proposed	Rationale
`RATE_CARD_MAX_AGE`	30 days	Cards drift as surcharges change. Past this, a locally computed quote is stale enough to mislead, and the buyer is told rather than quietly given an old price.
`FEED_POLL_MIN`	60 s	A floor on how often a client re-fetches a seller’s feed, so an eager implementation does not become the load problem.
`GRANT_LIFETIME`	24 h	Analytics disclosure grants (§13) expire by default. A grant that never expires is consent that cannot be withdrawn by inaction.
`CLOCK_SKEW`	±5 min	Inherited from the substrate. Timestamps are for display and ordering; where order must be authoritative it comes from a feed sequence, never a clock (§16.7).

19.7 Open

- **Whether `CONFIRM_TIMEOUT` is a protocol floor or an offer-level term** (§18.7). Listed here too, because the answer decides whether this table is normative for it or merely suggests a default.
- **Whether `MAX_REVIEW_BODY` should be very much smaller.** Its privacy purpose argues for something nearer a sentence than eight kilobytes; its usefulness argues the other way. The current value was chosen for the second reason and should be revisited for the first.
- **Every value in §19.2 and §19.3 is a proposal with no measurement behind it.** They are plausible and internally consistent, and they have never been tested against how real sellers and couriers behave. Recorded as such rather than presented with confidence they have not earned.

20. References

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

20.1 Scope and enforcement

Every standard this document profiles or intends to profile, split by whether an implementation must read it to be conformant (§20.2) or it supplies context and mapping targets an implementation does not need to

parse to interoperate (§20.3–§20.5). Each entry carries a "used for" column pointing at the section that relies on it, so a reference with no pointer is a reference that should not be here.

This completeness is partly machine-checked. The linter's T5 rule extracts every `RFC \d+` citation from the rest of the document and fails the build if it is absent from this section — an RFC cited in body prose but missing here is caught automatically. ISO standards, schema.org terms, GS1 identifiers, and the legal instruments in §20.4 have no such check: their completeness here is manual, and the same drift T5 exists to prevent for RFCs is possible for the rest of this table without anyone noticing.

20.2 Normative

Standards an implementation must read to be conformant.

Reference	Used for
RFC 2119 / RFC 8174 (BCP 14)	requirement-language keywords, interpreted only when written in capitals (§0.9)
RFC 8949 §4.2	deterministic CBOR encoding — the wire format every object in this document is serialised as (§1.3, §16.2)
RFC 9052	COSE structured signatures over CBOR objects (§1.3, §16.3)
RFC 8032	Ed25519 — the one signature scheme this document assumes (§1.3)
RFC 5545	`VAVAILABILITY` / `VFREEBUSY` payloads for time-slot availability, `RRULE` recurrence for capacity intervals and subscription periods (§3.4, §3.5, §5.6, §16.5.2)
ISO 3166-1	country and place codes (§4.9, §11.2, §13.4, §16.3)
ISO 4217	currency codes (§5.3, §16.3)
DMTAP substrate	Identity, Feeds & Blobs, Sync, Infrastructure Roles, Wake — the five capabilities TRACT adopts unchanged rather than reinventing (§0.3)

20.3 Informative — commerce and data-model standards

Context and mapping targets. An implementation interoperates without parsing these directly; a seller or gateway with an existing feed in one of these vocabularies uses them to translate in.

Reference	Used for
schema.org product vocabulary (`Product`, `Offer`, `ProductGroup`, `hasVariant`, `variesBy`)	the §2 data model, so existing merchant feeds map in by translation (§2.4)
GS1 GTIN / MPN	external product-identifier claims — advisory, unverified, squattable, never authoritative (§2.4)
GS1 SSCC	logistic-unit identification for physical consignments, where the parties already use it (§8)
Incoterms 2020	the risk/cost transfer point on shipped goods, distinct from the place-of-supply anchor it is easily confused with (§4.6, §4.9, §11.2)
UN/CEFACT, EDIFACT	legacy order/despatch/invoice mapping targets, for a seller whose counterparty is an ERP or a 3PL rather than another TRACT node (§7)
UPU conventions	cross-border postal-leg conventions (§8)
RFC 9458 (Oblivious HTTP)	an intended profile — not yet evaluated — for IP-level unlinkability at the transport, ahead of whatever the application-layer analytics grant already withholds (§13.7)

20.4 Informative — legal instruments

Named because §11 and §21.11's legal-grounding pass cite them for specific, narrow propositions. None of this is legal advice, and §21.11.5 lists the caveats that bound how far each reading travels — no case law has applied any of them to a permissionless no-operator protocol.

Reference	Used for
GDPR Art 4(1)	the working definition of personal data this document uses throughout, including a pseudonymous key linkable to a person (§0.9)
GDPR Art 17 (right to erasure)	the conflict between erasure rights and irrevocable content-addressed objects — unresearched after three passes, and the most likely hard blocker on the no-operator design (§0.5.1, §11.5, §21.11)
POPIA §1 (South Africa)	personal-data definition (§0.9); South African regime accommodation (§11.4)
LGPD Art 5 (Brazil)	personal-data definition (§0.9)
Cal. Rev. & Tax. Code §6041(b)	the contract-gated US marketplace-facilitator test — the strongest structural argument for a gateway no seller has contracted with falling outside the definition (§11.2a, §21.11.1)
Wash. RCW 82.08.010(15)(a)	the three-prong US facilitator test, all three required simultaneously per the state's own guidance (§11.2a, §21.11.2)
N.Y. Tax Law §1101(e)(1)(B)	a facilitator trigger met by contracting with a third party to collect receipts, catching a gateway whose checkout merely routes through its own payment provider (§11.2a, §21.11.2)
Tex. Tax Code §151.0242(a)(2)	a facilitator trigger for directly or indirectly processing sales or payments, with no payment-processor carve-out (§11.2a, §21.11.2)
Council Directive 2006/112/EC Art 14a (EU VAT Directive)	deemed-supplier scope for imported consignments of intrinsic value ≤ €150 and intra-EU supplies by non-EU-established sellers (§11.2b, §21.11.3)
Council Implementing Regulation (EU) 282/2011 Art 5b	the cumulative not-facilitating test and the "economic reality and influence" standard, which the Commission's Explanatory Notes read as rejecting a purely contractual escape (§11.2b, §21.11.3)

20.5 Informative — research literature

Cited for one specific, checked claim each — that a mechanism exists and has shipped — not as a general endorsement of the surrounding paper.

Reference	Used for
O'Neil, "The Escrow Transactional Method" (1986)	the origin of the escrow/demarcation family the bounded-counter inventory mechanism descends from (§6.2a, §21.10.1)
Barbará-Millá & Garcia-Molina, "The Demarcation Protocol" (1994)	the demarcation protocol that family passes through on the way to a CRDT (§6.2a, §21.10.1)
Balegas et al., SRDS 2015	`BoundedCounter`, shipped in production as AntidoteDB's `antidote_crdt_counter_b`, and the measured ~200-decrement oversell in a plain eventually-consistent counter at 200 concurrent clients that it corrects (§6.2a, §21.10.1)

20.6 On standards reuse

TRACT invents only where no standard exists. Where a proven specification fits, this document **profiles** it rather than restating it — and where it profiles, the referenced standard governs its own bytes.

20.7 Open

- Whether the informative tables need version pinning the way RFCs are immutably numbered. Incoterms is revised on a roughly ten-year cycle and §4.6 currently names “Incoterms 2020” without a stated policy for what happens to a signed offer when Incoterms 2030 exists.
- Whether T5’s completeness check should extend beyond RFCs to ISO standards and GS1 identifiers, which have the identical drift risk and none of the automated protection (§20.1).
- Whether the legal-instrument table (§20.4) needs a “checked as of” date given §21.11.5’s own caveat that Washington already amended its definition once in 2026 and ViDA will change the EU scope — a reference list of statutes is not append-only the way a reference list of RFCs is.
- Whether the DMTAP substrate row needs to name a specific version or commit once the substrate specification itself stabilises, rather than a bare directory reference.

21. Grounding — what the evidence actually supports

> **Why this section exists.** Every claim in this specification about what decentralized commerce can do is a claim about the future. The claims below are about the past, and they are less flattering. This section records what a 107-agent, adversarially-verified literature pass found in July 2026, including the findings that **contradict** parts of this document, so that a reader can tell the difference between a design decision and a demonstrated result.
>
> Nothing here is normative. Everything here should change what is normative.

21.1 Coverage — read this before citing anything below

Two passes have run. Between them, **four areas produced claims that survived three-vote adversarial verification:** deployed networks, global product identity, the goods/variant slice of the data model, and — from the second pass (§21.10) — live commerce state.

Still unresearched after both passes, and therefore neither supported nor refuted: logistics standards and consolidation optimisation (§8); privacy-preserving analytics and fraud (§13); and the whole of trust, dispute, tax and legal (§9.6, §10, §11). The second pass targeted all three directly and returned **nothing verified** on any of them, which makes them a standing gap rather than an oversight.

Those gaps are **absence of evidence, not evidence of absence**, and §8, §10, §11 and §13 must not cite this section as support. §11 in particular currently answers by assertion — who the marketplace facilitator is when there is no marketplace operator, and how erasure rights are satisfied against irrevocable objects, are both unanswered here. Within the covered areas, several named systems also produced nothing verified: Particl, Origin Protocol, BOLT12, x402, L402, and ActivityPub-based commerce.

21.2 The finding that most threatens this design

> **There is no deployed system achieving cross-publisher product identity without a licensed registry, and the one candidate for a permissionless alternative was refuted.**

The two deployed models are the only two:

Model	Property	Cost / gate
GS1 GTIN	permissioned monopoly namespace	*licensed*, not sold; issued through national member organisations; US$30 per identifier at GS1 US (verified 2026-07-21, US-specific — do not generalise). The free tier is explicitly restricted to closed/local circulation.

Model	Property	Cost / gate
schema.org `productGroupID`	purely nominal	a plain `Text` string — no issuer, no registry, no uniqueness guarantee

There is nothing in between. A claim that crawl-derived clustering over schema.org identifiers demonstrates feasible permissionless product entity resolution was **refuted 0-3**. So this document contains **no positive evidence that cross-publisher product deduplication works at scale.**

What this means for §2. The content-address floor is sound as a *mechanism* — identical bytes converge, trivially. It is unproven as a *solution*, because independent publishers describing the same physical product do not produce identical bytes, and nothing here shows that the gap can be closed adversarially. §2's canonicalisation rules are therefore the load-bearing part of that section, not the addressing, and §2.5 must keep near-duplicate resolution listed as **open** rather than implied-solved. Language suggesting a global product view "falls out of" content addressing overstates what is known.

21.3 The OpenBazaar postmortem

OpenBazaar was the closest deployed relative of this design: signed objects, content-addressed listings, keypair identity, no operator. It shut down in January 2021. Cite it as a postmortem, never as a live comparison.

Measure	Result
Lifetime participants	~6,651
Concurrently online	~80
Credible sales, 14 months	~US$86,000
Median listing lifetime	~22 days
Share of measured sales value faked by one vendor	**60%**

Four failure modes, each of which maps onto a section of this document:

1. **Negligible economic activity** — orders of magnitude below centralized equivalents.
2. **Discovery re-centralized first.** A content-addressed substrate offers no global index, so search was outsourced to crawler operators, and the default one became a content-policy gatekeeper. → §2.6.
3. **Availability was bounded by publisher liveness.** Whole catalogues disappeared when a merchant node went offline; third-party indexes went stale against the live network. → §21.5.
4. **Reputation was trivially ballot-stuffed and structurally unenforceable**, and **opt-in escrow and moderation went unused precisely where they mattered most** — bad actors simply declined them. → §21.6.

Source caveat: all four rest on one peer-reviewed, DHS-funded measurement paper and share its methodology. Sales figures are an author-acknowledged lower bound from voluntary feedback, and item-survival numbers conflate deliberate delisting with liveness eviction.

21.4 What the largest live network chose instead

Beckn / ONDC is the biggest deployed "decentralized commerce" network, and it avoids OpenBazaar's failure modes by doing the opposite of this specification:

- **Registry-mediated, not self-certifying.** Participants register keys via `POST /subscribe`; counterparties resolve endpoints and public keys through a registry lookup — not gossip, not a DHT, not content addressing.
- **Approval-gated permissioned enrollment.** Portal whitelisting (6–48h), a signed agreement, certification, and probation before a participant may transact.

- **Identity anchored to DNS/TLS domain names, not keypairs** — inheriting CA and DNS centralization on top of registry centralization.
- **Discovery through operator-hosted, rate-limited lookup endpoints**, giving the operator a throttle and deny lever over network-wide discovery.

This is the honest comparison, and it is uncomfortable. The network with volume chose an operator at exactly the three points this document tries to leave operator-free. That is not proof this design fails; it is evidence that the burden of proof sits here, not with them.

Time sensitivity: Beckn developer docs are stamped 2021; ONDC has deprecated `/lookup` and `/vlookup` in favour of `/v2.0/lookup`, whose rate limit is published as "TBD".

21.5 Liveness is a first-class problem, not an edge case

§0.7 says durability lives at the edges and the sender's node retries. That is true **for orders** and false **for catalogues**.

A seller whose node is offline is not merely slow to receive orders — they are **invisible**. OpenBazaar's ~22-day median listing lifetime and mass catalogue disappearance on node departure are the measured consequence. §21.1 notwithstanding, this one is directly evidenced.

Implications this document must carry rather than bury:

- Documentation MUST NOT imply that an intermittently-online seller is fully functional. A laptop that sleeps can *receive orders* through the substrate's mailbox and wake path; it cannot *serve a catalogue* while asleep.
- Third-party caching/pinning of public catalogue objects moves from a nice-to-have to a requirement for any seller not running an always-on node — and **unpaid replication is exactly what OpenBazaar did not get**. Whether pinning needs an incentive, and whether that incentive creates another operator, is open.
- A gateway serving a store is, incidentally, a liveness backstop. That is an argument *for* the operator class of §0.4.2, and it should be counted honestly as one.

21.6 Reputation: the documented failure mode is this design's

OpenBazaar's reputation failure was **self-published, unweighted reviews with no banning authority**, combined with **opt-in escrow that bad actors declined**. One vendor faked 60% of measured sales value.

§10 proposes purchase-attested reviews, which is strictly stronger than what OpenBazaar had — an attestation binds a review to a transaction. But two of OpenBazaar's failure conditions survive unchanged in this document as currently written:

1. **Self-dealing.** A seller transacting with itself produces genuine attestations. Attestation raises cost; it does not establish that the counterparty was independent.
2. **Opt-in escrow is declined by exactly the actors it exists to constrain.** §9.6 makes escrow per-order and optional, and §9.6's own reasoning — that mandatory escrow would exclude underserved regions — is sound. But the measured consequence of optionality is that it goes unused where it matters. Both cannot be true costlessly.

§10 and §9.6 must state this rather than claiming attestation resolves it. The sybil-cost floor achievable on a signed-feed substrate is listed in §21.8 as an open question because the attack literature was not reached by this pass.

21.7 A warning about scope

Nostr — a signed-feed ecosystem structurally similar to this substrate — marked its marketplace specification **NIP-15 "unrecommended: too complicated"** in-repo (banner added 2026-05-31) and redirected implementers to NIP-99, a far simpler classified-listing primitive. Neither solves "many stores, one SKU": product IDs are merchant-generated and merchant-scoped, discovery is free-text tags, and there is no cross-publisher product identifier.

NIP-15 also moves the entire order and checkout flow off the public feed into encrypted messages with **the merchant as sole authority** asserting payment and shipment — no escrow, no buyer counter-signature, no third-party-verifiable order state. This document's §7 signed-transition model is a deliberate departure from that, and the departure is the point; but the ecosystem's own verdict on structured-commerce complexity is a caution this specification should not ignore.

Wording note: the repo status word is "unrecommended", not "deprecated".

21.8 Open questions this pass could not close

1. **Does any deployed system achieve cross-publisher product identity without a licensed registry?** The design space between GS1 monopoly and nominal merchant string is currently evidence-free. Needs a dedicated pass on entity resolution, blocking strategies and probabilistic matching **under adversarial publishers** — including deliberate collision.
2. **What is the achievable sybil-cost floor for reputation on a signed-feed substrate?** Do buyer counter-signatures, transaction-cost binding, or local-only web-of-trust scoring actually resist ballot-stuffing? Entirely unresearched.
3. **Where does a spec put the operator-shaped functions it cannot eliminate — indexer, registrar, arbiter, pinner?** ONDC answers "one central approval-gating registry". OpenBazaar answered "nowhere" and got a gatekeeping default search engine plus liveness-bounded availability. **Is there a middle design** — multiple competing indexers with verifiable completeness or censorship proofs, federated or rotating registrars, paid pinning markets — **with any deployed precedent and measured outcomes?** This is the central unanswered question for §0.4.
4. **How do erasure rights and identifiable-seller mandates interact with immutable, content-addressed, replicated commerce objects?** Who is the data controller when order data is replicated across independent nodes; how is erasure satisfied against irrevocable objects; who satisfies trader-traceability duties when identity is a keypair; and who is the "marketplace facilitator" for tax purposes when no marketplace operator exists. **These are the most likely hard blockers on a no-operator design**, and §0.5.1 and §11 currently answer them by construction and by assertion respectively, not by evidence.

21.9 What this section obliges

- §2 must not imply that content addressing solves global product identity (§21.2).
- §2.6 must state that "any node MAY build an index" does not prevent one index becoming the de facto gatekeeper (§21.3, §21.4).
- §0.7 and all self-hosting guidance must distinguish receiving orders from serving a catalogue, and must not present an intermittently-online seller as fully functional (§21.5).
- §9.6 and §10 must state the opt-in-escrow and self-dealing failure modes as measured outcomes, not hypotheticals (§21.6).
- No section may cite this one as support for logistics, analytics, live-state or legal claims (§21.1).

21.10 Second pass — live commerce state (July 2026)

A second adversarially-verified pass covered the four areas §21.1 records as unresearched. **Only one of them produced surviving claims: live commerce state.** Logistics and delivery, privacy- preserving analytics and fraud, and the whole of trust/dispute/tax/legal produced **no verified findings and remain unresearched** — §21.1's prohibition on citing this section for them still stands, and §8, §10, §11 and §13 are still unevidenced.

21.10.1 The good news: no-coordinator oversell prevention is real

A seller running N replicas **can** guarantee no oversell with no coordinator. The mechanism is the escrow/demarcation family — O'Neil (1986) → Barbará-Millá & Garcia-Molina (1994) → Balegas et al., SRDS 2015 (`BoundedCounter`), shipped in production as AntidoteDB's `antidote_crdt_counter_b`.

The gap between the counter and its bound is pre-partitioned into transferable **rights**. A replica may decrement only from rights it holds locally, and the locally computed right count is always conservative — it can under-count but never over-count. So the invariant holds **even from stale state**, and the guarantee survives arbitrary message loss, delay, reorder and partition.

The contrast matters: a plain eventually-consistent read-check-decrement counter measurably **does** oversell — roughly 200 excess decrements at ~200 concurrent clients in the SRDS'15 experiment. This is not a theoretical risk being designed around.

The guarantee is safety-only and is paid for entirely in liveness: stranded quota, spurious "sold out", and aborted decrements while global stock remains. §6.4 already states that residual.

21.10.2 The bad news: four operator-shaped roles inside the counter

Each of these must be accepted or replaced explicitly by §6. None was visible before this pass.

1. **An intra-replica serialization point is mandatory.** A single-writer node, a compare-and-swap, or consensus. **A "replica" therefore cannot itself be a set of mutually uncoordinated nodes** — the coordination is not removed, it is pushed down one level and made local.
2. **The failure model is crash-recovery with durable state, not Byzantine.** This is the sharpest finding in the pass. **A lying or state-losing replica simply oversells**, and none of this literature covers stock held across *mutually distrusting* parties. The mechanism is sound for a seller's own replicas, which trust each other; it says nothing about a seller who lies.
3. **Reclaiming a permanently-dead replica's rights requires an agreement or failure-detector decision** — and if that decision is wrong, rights are double-issued and the invariant breaks. Deciding a peer is dead is exactly the kind of judgement a coordinator makes.
4. **With three or more replicas, transfer is no longer anonymous.** The giver must name the recipient, which reintroduces an allocation policy — and the papers themselves suggest centralising it.

21.10.3 Sagas do not provide what multi-party checkout needs

Confirmed rather than assumed: sagas explicitly provide **neither atomicity nor isolation**. Other parties observe partial sagas, and there is no abort cascade. So multi-party checkout gets eventual amendment, never oversell prevention — **oversell prevention has to sit in the per-replica invariant primitive**, not in the checkout flow above it. §6.4's "no cross-seller atomicity" is therefore not a limitation to be engineered away later; it is the correct shape.

21.10.4 What §6 must now say

- Name the intra-replica serialization requirement rather than implying replicas need no coordination at any level (§21.10.2 item 1).

- State the **non-Byzantine failure model as a hard boundary**: bounded counters protect a seller from their own concurrency, not a buyer from a dishonest seller. Conflating the two would be a safety claim the literature does not support (item 2).
- Specify dead-replica right reclamation as an explicit, fallible decision with a stated failure mode, not a background detail (item 3).
- Acknowledge that transfer names a recipient at N ≥ 3, and say who chooses (item 4).

21.11 Third pass — who is the facilitator (July 2026)

A narrow pass on the five legal questions §11 answers by assertion. **One of the five produced verified findings. The other four returned nothing across three consecutive passes** and are unresearched, not resolved: GDPR Art 17 against immutable objects, DSA/INFORM/GPSR trader traceability, escrow as a regulated payment service, and measured outcomes of non-custodial escrow. Their silence is not a favourable answer.

Everything below is statutory construction and regulator guidance. **No court anywhere has applied any of it to a permissionless no-operator protocol.** The favourable readings are textually strong and legally untested, and that gap is the finding as much as the readings are.

21.11.1 The favourable finding, and exactly how far it reaches

Every representative US state marketplace-facilitator test is **conjunctive and gated on a contract with the seller**. Cal. Rev. & Tax. Code §6041(b) reads "a person who **contracts with** marketplace sellers to facilitate … through a marketplace **operated by the person** … **and** who does both of the following", and its infrastructure prong — "owning or operating the infrastructure … that brings buyers and sellers together" — sits *inside* that contract gate. Wash. RCW 82.08.010(15)(a) requires all three of contract, transmission of offer/acceptance, and a listed activity; the state's own guidance says "all three main criteria must be met simultaneously".

So a gateway no seller has contracted with falls outside the definition on its face, whatever infrastructure it runs. That is the strongest structural argument the no-operator design has, and it is untested.

21.11.2 Where the argument does *not* reach — three corrections §11 must absorb

The escape is at the facilitator prongs, not at "marketplace". The term is deliberately medium-agnostic and expressly enumerates *a catalog* and *a dedicated sales software application*. TRACT's signed catalogue feed and its buyer-side cart client are **within** the definitional term even where no person qualifies as a facilitator. §11 may not argue "there is no marketplace"; the available argument is only "there is no facilitator".

Escrow is the decisive practical trigger, and in two states it is load-bearing on its own. Tex. Tax Code §151.0242(a)(2) catches a person who owns or operates a marketplace and "**directly or indirectly processes sales or payments**", with no payment-processor carve-out — a gateway receiving buyer funds and releasing them is at minimum indirectly processing. N.Y. Tax Law §1101(e)(1)(B) is met where the person "**or contracts with a third party to collect**" the receipts, so a gateway whose checkout routes through its own PSP is caught even though its balance sheet never holds the money. In Washington and California payment is a qualifier rather than a standalone trigger.

"Render-only never touches funds" is not a US answer. It is safe in New York and Texas and likely caught in Washington and California through the listing and order-taking prongs. A spec resting on render-only as a non-facilitator posture is resting on two states out of fifty.

One assertion in §11.3 is confirmed: a self-hosted seller taking direct payment for their own goods is never a marketplace facilitator, because every marketplace definition requires sales by persons *other than* the owner or operator of the medium.

21.11.3 The adverse finding: EU VAT anticipates this design's central claim

This is the one that should change how §11 is written. The binding text is Art 5b of Council Implementing Regulation (EU) 282/2011: an interface is **not** facilitating only if, *cumulatively*, it (a) does not set, directly or indirectly, **any** of the terms and conditions; (b) is not, directly or indirectly, involved in authorising the charge; and © is not, directly or indirectly, involved in the ordering or delivery. The Commission's Explanatory Notes state that carrying out **even one** of those may suffice to make an interface a facilitator.

And then, verbatim, the sentence that names TRACT's argument and rejects it:

> "The use of 'indirectly' and 'any' … is meant to prevent artificial splitting of rights and obligations between the electronic interface and the underlying suppliers. For example, the indication that the seller … is responsible for the goods sold via a marketplace/platform **or that the contract is concluded between the underlying supplier and the customer is not sufficient** to relieve the taxable person operating the electronic interface from the VAT obligations as a deemed supplier. The concept thus goes beyond the contractual relationship and looks at the **economic reality** and in particular the **influence** exercised by … electronic interfaces."

"The contract is between two keypairs" is precisely the assertion this text was written to defeat. The rule tests economic reality and influence, not what the parties declare — and a protocol asserting a contractual arrangement is, on this reading, exactly the artificial splitting the words "indirectly" and "any" exist to catch.

The scope limit that keeps this survivable: Art 14a bites on imported consignments of intrinsic value ≤ €150 and on supplies within the EU by non-EU-established sellers. It is not a general rule for all EU commerce. But where it applies, no amount of protocol design argues its way out.

21.11.4 What §11 must now do

- **Stop arguing there is no marketplace.** There is; the argument available is that there is no facilitator, and only where no seller contracted with the gateway.
- **Say that escrow is the trigger**, and that in Texas and New York it is enough on its own. §9.5's "escrow is an operator class" understates it: escrow is also the thing most likely to make that operator a *tax* facilitator.
- **Stop presenting render-only as safe.** It is a two-state holding.
- **State that the EU VAT rule anticipates the design's central claim** rather than leaving a reader to discover it, and give the scope limit honestly rather than as reassurance.
- **Keep the four unresearched questions marked as unresearched** (§21.11), especially the GDPR erasure conflict, which remains the most likely hard blocker and has now survived three passes without an answer.

21.11.5 Caveats that bound all of the above

No case law anywhere. Four US states of roughly fifty, and they diverge materially. **DAC7 is unresolved in both directions** — every DAC7 claim in this pass was voted down. The EU VAT finding rests partly on Explanatory Notes that disclaim binding force, over an Art 5b text the CJEU has never interpreted. Washington amended its definition in 2026; ViDA will expand EU deemed-supplier scope; states have shown willingness to deem specific verticals in by name. **A currently-true textual escape is not a structural guarantee**, and this section should be re-checked rather than cited indefinitely.

22. Erasure — the hardest unresolved problem in this design

> **Drafting status.** This section is scoped but not yet normative. It states what it will specify, which existing standards it profiles, and the decisions still open. Nothing here is implementable yet; text becomes normative when the RFC 2119 keywords appear.

22.1 Scope

What happens when a data-protection erasure right — GDPR Art 17, POPIA section 24, LGPD Art 18 — is asserted against a TRACT object, or against the fact that one was ever published. This section does not resolve that question. It states it precisely, works through the mechanisms available for the residual once the structural answer is applied, and names what is still open after three research passes found nothing (§21.11).

22.2 The conflict, stated precisely

A published TRACT object is content-addressed: its address is a hash of its bytes, and any node that holds those bytes may serve them to anyone who asks, forever, without asking the publisher first (§0.4.1, §2.6). That is not an accident of implementation — it is the entire mechanism by which two sellers' catalogues converge, an index rebuilds itself from nothing, and a cache or a pin keeps a sleeping seller's storefront answerable (§1.7, §21.5). The same property that makes the network usable without an operator makes the object impossible to recall once it exists.

"Delete it" is not available against an object like that, for a structural reason rather than a policy one: there is no single custodian to instruct. A conventional erasure request names a controller who holds one copy in one system and can remove it. Here, by design, the set of parties who may hold a copy is unenumerable — any cache, any pin, any index that snapshotted the feed, any buyer's client that fetched it before a retraction, any node nobody involved in the request has ever heard of (§0.4.1's cache/pin role requires nothing but willingness to hold bytes; there is no registry of who has). A right that presupposes a reachable custodian has no target here that can satisfy it the way the statute contemplates. This is not a claim that the right does not apply; it is a claim that the object it would be asserted against was built to survive exactly the kind of removal the right asks for.

22.3 Why the structural answer is the primary defence

§0.5.1 does not attempt deletion. It refuses the premise: **no personal data enters the public quadrant at all.** An order carrying a name, address or contact detail is sealed, held at the two endpoints, and deletable there in the ordinary sense — because there, for once, the custodian is countable (§0.5, §16.6). The public quadrant is grammatically incapable of carrying the fields an erasure right would ever be asserted against (§16.4): there is no street-address production in the public object family, no name field on a product record or offer, nothing to redact because nothing personal was ever admitted.

This is a stronger position than any deletion mechanism could offer, and the reason is worth stating plainly: a system cannot be asked to erase what it never published. Deletion mechanisms are always reactive — they reach into a system that already let something escape and try to remove it, against every copy that may already exist. Structural exclusion is not reactive; there is nothing that escaped, because the grammar never accepted it. §16.4 exists precisely so this is enforced by what a decoder can encode, not by a publisher's discipline or a reviewer's attention — an object that structurally cannot carry a name cannot leak one by someone forgetting a check.

The defence is complete only where it is total, and that is the honest qualifier. §0.5.1's value depends on there being no exception, because a single field that lets personal data into a public, irrevocable object is not

"mostly" fixed by a later mechanism — the object is irrevocable from the moment it is published, and a residual mechanism applied afterward inherits every limit described below. §0.5.1 has exactly one acknowledged exception, and this section exists because of it.

22.4 Candidate mechanisms for the residual

None of the three mechanisms below is a solution. Each is a partial answer to a narrower question, and each has a limit that does not go away with better engineering.

Mechanism	What it actually does	What it does not do
Crypto-shredding	destroys future ability to decrypt, for everyone, once the key is gone	does not remove the ciphertext bytes, and does not fit the public quadrant's plaintext-serving model
Tombstone / supersede	tells conformant participants to stop showing the object	binds only participants who look for it and choose to honour it
Off-chain payload	removes the content at its source once the source is willing	leaves the address, and the address is not nothing

22.4.1 Crypto-shredding

The idea: publish ciphertext at a content address instead of plaintext, keep the decryption key off the public feed, and destroy the key when erasure is wanted. Once the last copy of the key is gone, no holder of the ciphertext — however many independent holders there are — can ever recover the plaintext again. That is a real and useful property, and it is the only one of the three mechanisms that can, in principle, make previously-published content permanently unreadable rather than merely harder to find.

It does not fit cleanly into TRACT as designed. The public quadrant exists to be computed over — canonicalised, matched, indexed, ranked (§2.2, §2.6) — and a ciphertext blob nobody but the key-holder can read is not a catalogue entry anyone can index; it defeats the reason objects are public in the first place. Applying crypto-shredding to product records, offers or rate cards would mean publishing objects that are structurally public but functionally private, which is a category neither §16.4's two type families nor §2's canonicalisation rules were built to hold. On the sealed side — orders — the mechanism is not needed: both endpoints already hold their own copy and can delete it directly (§0.5, §16.6); there is no third party's inaccessible ciphertext to worry about, because sealed objects were never replicated past two parties to begin with.

Where the question is not academic is the general one, independent of whether TRACT ever adopts the mechanism: **does destroying the key satisfy an erasure right, or does it only make the data permanently inaccessible without deleting it?** Those are not the same claim. A regulator or court could read "erasure" as requiring the referenced data cease to exist, in which case a permanently unreadable ciphertext blob sitting at a stable address does not satisfy it, however inaccessible it is. Or it could accept irrecoverability as functionally equivalent to erasure, on the reasoning that data nobody can ever read again is not meaningfully different from data that does not exist. This document takes no position, because none of the three research passes behind §21 found an answer either way, and the question has never been tested against a network where the ciphertext itself is replicated beyond the publisher's reach by design (§21.11). It is recorded here as unresolved, not because it was overlooked, but because it was looked for and not found.

22.4.2 Tombstones and supersede markers

TRACT already uses this pattern twice: an offer that no longer stands is superseded, not deleted (§18.2), and a review's retraction is a superseding tombstone honoured by conformant clients and gateways (§10.2, §10.4). The mechanism is a later signed object that says "the thing at this address no longer represents the publisher's current position," and every participant that checks for one before serving or displaying the original object will show the tombstone instead.

"Cooperative-only" is the honest description, and it means exactly this: the tombstone changes what a conformant implementation shows. It changes nothing about what bytes exist. A holder that serves whatever is at an address without checking the feed head for a later entry, an index that snapshotted before the supersede was published, a cache that pinned the object independently of the feed it came from, or a party running software that simply does not implement retraction — none of these is violating the protocol, because nothing in the wire format obliges a byte-holder to notice a later message, and nothing can make it notice one. §18.2 already states this for offers — "withdrawal is not deletion" — and the same sentence applies here without qualification: a tombstone is a request honoured by cooperation, not a capability enforced by construction. Where §0.5.1's grammar-level prohibition is strong because it never lets the object into existence, a tombstone is weaker in exactly the way any after-the-fact mechanism is weaker than never publishing at all.

22.4.3 Off-chain or out-of-band payload

The idea: keep only the address in the public, replicated part of the system, and hold the actual payload somewhere the publisher controls directly — a private server, a bucket with access control — so that removing the payload at its source means the address no longer resolves to anything new.

This does not sit comfortably inside a content-addressing model, where the address *is* the hash of the content: moving the payload without changing the address requires a level of indirection — a pointer object that resolves an address to a location rather than the content itself — that TRACT does not otherwise have, and introducing one reintroduces exactly the kind of authoritative, centrally-resolved reference this design avoids everywhere else. That tension is not resolved here.

More importantly, **the address itself is not nothing**, even once the payload behind it is gone:

- **It proves an object once existed at that address**, and anyone who cached the payload before it was withdrawn still holds it — the address does not un-happen, and any independent holder from §22.4.2's discussion above may still be serving the bytes regardless of what happens at the source.
- **It is a join key wherever it is referenced elsewhere.** A product record's `group` or `components` fields, a review's `Subject`, a purchase attestation's order reference (§16.5.1, §16.5.5) all carry content addresses forward into other public objects. Removing a payload does not remove the addresses that point at it from objects that are themselves still public, so anyone holding both the reference and a cached copy of the referenced payload can still correlate them.
- **Feed position and timing leak on their own.** A feed's sequence and timestamps are public regardless of what the referenced payload said (§16.7), so the fact that some object existed, was published by a given key, at a given time, and was later withdrawn, is itself potentially identifying information that removing the payload does not touch.

22.5 Who is the controller

A sealed order exists at exactly two places: the buyer's node and the seller's node, each typically a self-hosted box belonging to a different natural person (§0.4.1 — a seller needs nothing rarer than "a keypair and a box that is up"). Data-protection regimes built around a business processing customer records assume a controller with an organisational identity distinct from the data subject. Here that assumption may not hold, and this document does not know the answer to either question it raises:

- **Is this joint controllership?** GDPR-style regimes have a concept for two or more parties who jointly determine the purposes and means of processing the same data, with obligations that follow — a transparent arrangement setting out each party's responsibilities being one of them. Whether that is the right frame for two strangers who "have never heard of each other" (§0.1), transact once, and each independently decide to create and retain their own copy of a shared sealed object, is not something this document asserts either way. It may be the correct legal category with no practical way to discharge its own

duties between parties who never negotiate anything else about the trade; it may be the wrong category entirely. Both readings are guesses and are labelled as such rather than adopted.

- **Does a household-style exemption apply to a self-hosted seller?** A regime that exempts purely personal or household activity from its scope was not written with a keypair-identified seller running their own node for their own trade in mind. Where the boundary of that exemption sits for someone who is, in substance, conducting a commercial activity but has no formal business registration, no employees, and no infrastructure beyond their own machine, is not answered here. It is not answered anywhere this document's research reached (§21.11).

Neither question was resolved by the pass that looked for an answer to who is a "facilitator" (§11.2a, §21.11) — that pass covered the gateway's position, not the two ordinary parties to a sealed order, and nobody has separately researched this one.

22.6 Reviews — the one bounded exception, worked through

§10.4 already names the residual; this section works through why it exists and what it does not close. A review is the one object in the public quadrant that is, by design, signed by a natural person and meant to be recognisably the same person across their own history with one seller — the entire value of a review is that it is attributable to a real purchaser, which is in direct tension with keeping the author unidentifiable.

TRACT's mitigation is the per-subject pseudonymous subkey (§1.5, §16.5.5): `Review.author` is never the buyer's root `IK`, but a key derived for, or generated for, that one seller relationship. Two sellers comparing their reviews cannot join them by author key; only the buyer, holding the root key or the stored mapping, can compute the correspondence. Retraction follows the same supersede pattern as everything else public (§18.2, §10.2): a later signed tombstone, honoured by conformant holders, with exactly the cooperative-only limit described in §22.4.2 — bytes persist wherever an independent holder kept them, and that residual has to be disclosed to the author **before** they publish, not discovered afterward (§10.4).

One point this document's own glossary already forces and is worth making explicit here: **a pseudonymous key linkable to a person is still personal data** (§0.9). Pseudonymisation lowers risk; it does not remove the datum from scope. This matters twice over for the review subkey specifically. First, under deterministic derivation (§1.5), the unlinkability was never structural — it was the difficulty of a computation that is zero for whoever holds the buyer's root key, which means a compelled disclosure or a seized device recovers every per-seller subkey the buyer ever used, not just this one review's author key. Second, even under the stored-random alternative, the subkey remains, by the glossary's own definition, personal data for as long as the buyer — the only party who can compute or has stored the correspondence — can be identified by it. Neither derivation choice turns the review into something outside scope; both only change who can perform the linkage and how expensively.

22.7 What a deployment can do today

None of this waits on a legal answer. Several things reduce the residual regardless of how §22.5 and §22.4.1's open question eventually resolve, and a deployment can act on them now:

- **Publish less.** §16.4's grammar already refuses names and addresses in the public family; nothing in the free-text fields it cannot constrain — a product description, a review body — has to be filled with more than it needs. The grammar cannot stop a person typing an address into `Review.body` (§16.5.5 says so plainly); a client's own UI can warn before it lets them.
- **Keep the review body small.** `MAX_REVIEW_BODY` is set at 8 KiB (§19.5), a limit stated there as serving privacy as much as resource bounding — the smaller the field, the less room there is to put identifying detail into a public, irrevocable object. §19.7 already flags this value as probably too large for the privacy

purpose it is supposed to serve, chosen instead for usefulness, and recommends revisiting it in the other direction. This section agrees with that flag rather than repeating it as a separate open item.

- **Prefer recoverable unlinkability over recoverable identity**, where an implementation has the choice §1.5 leaves open: a stored-random subkey mapping costs recovery convenience (losing the mapping severs continuity with a seller) but does not hand a compromised root key the ability to retroactively join a buyer's entire purchase history; a deterministic subkey costs exactly the reverse. Neither is mandated; a deployment can pick the trade-off that matches what it is more afraid of.
- **Disclose irrevocability before the act, not after.** This document already applies that rule elsewhere — a non-custodial rail's dispute deadlock has to be disclosed before the trade (§18.5), and a review's residual has to be disclosed to its author before publishing (§10.4). The same discipline generalises: any interface about to publish something into the public quadrant should say, at the moment of the action, that what is about to be published cannot be fully taken back — not surface that fact only once someone asks for it to be removed.

None of this closes the gap in §22.5 or resolves the open question in §22.4.1. It shrinks the residual that those unanswered questions would otherwise be asked to cover.

22.8 Open

This is, in the judgement of this document, the most likely hard blocker on the whole design, and the list below is left long deliberately rather than tidied into false confidence.

1. **Whether destroying a crypto-shredding key satisfies a statutory erasure right, or only makes the referenced data permanently inaccessible without deleting it** (§22.4.1). Unresolved; not found by any of three research passes (§21.11).
2. **Whether joint controllership is the right frame for a sealed order held at two independently self-hosted nodes**, and if it is, how its arrangement duties could ever be discharged between two parties who transact once and have no other relationship (§22.5).
3. **Where, if anywhere, a household-style exemption applies to a self-hosted seller** with no formal business registration and no infrastructure beyond their own node (§22.5). Unresearched, not merely unsettled.
4. **Whether `MAX_REVIEW_BODY` should shrink further**, trading usefulness for a smaller residual, as §19.7 already flags and §22.7 endorses without deciding by how much.
5. **Whether a supersede/tombstone mechanism should ever be given protocol-level teeth** — a propagation obligation on indexes and caches, for instance — or whether that is permanently out of reach because enforcing it requires exactly the coordinating authority this design refuses to introduce anywhere else (§22.4.2).
6. **Whether crypto-shredding has any legitimate place inside TRACT's model at all**, given that the public quadrant exists specifically to be served and computed over in plaintext, or whether it only ever applies as voluntary defence-in-depth on the sealed side, where the problem it would solve does not exist because both endpoints can already delete their own copy (§22.4.1).
7. **Whether trader-traceability duties that require identifying a seller** (§11.4) collide with minimising what is public, and if they do, which one this document should bend — that tension is named, not resolved, here.
8. **This section needs legal advice, not another literature pass.** Three research passes have now looked at the GDPR erasure conflict specifically and returned nothing verified each time (§21.11). That is not a gap this document can close by writing more carefully about it; it is a gap that closes, if it closes, with an opinion from someone qualified to give one in each jurisdiction this protocol is deployed into.

Drafted with the help of an LLM by Imran Yusuf Paruk · Durban, South Africa